TRIBHUVAN UNIVERSITY
FACULTY OF HUMANITIES
AND
SOCIAL SCIENCE



Project Report

on

SignSathi: Real Time Nepali Sign Language Recognition App

For the partial fulfillment of the requirements of the degree of Bachelor of Computer Application

Submitted By:

Arjun Raj Acharya (54602068)

TU Registration No. : 6-2-546-24-2021

Asha Mohan Tamang (54602069)
TU Registration No. : 9-2-224-4-2020

Submitted To:
Department of Computer Application
Tribhuvan University
Kathmandu

Under the supervision
of
Santosh Neupane

October 2026

SUPERVISOR RECOMMENDATION

This is to certify that the final year project entitled **"SignSathi: Real Time Nepali Sign Language Recognition App**" is an academic work completed by **Arjun Raj Acharya(54602068) and Asha Mohan Tamang(54602069)** submitted in the partial fulfillment of the requirements for the degree of Bachelor of Computer Application awarded by Tribhuvan University under my guidance and supervision.

The information presented by him/her in the project report has not been submitted earlier to the best of my knowledge,

_____

Mr. Santosh Neupane
Project Supervisor/ Lecturer
Department of Computer Application

Xavier International College
Date: October 2026

CERTIFICATE OF APPROVAL

The undersigned certify that they have read and recommended to the Department of Computer Application Xavier International College for acceptance, a Final year Project Report entitled **"SignSathi: Real Time Nepali Sign Language Recognition App**" submitted by **Arjun Raj Acharya(54602068) and Asha Mohan Tamang (54602069)** for the partial fulfillment of the requirement for the degree of Bachelor of Computer Application awarded by Tribhuvan University.

……	……
Mr. Santosh Neupane	Mr. Rajesh Nepal
Project Supervisor/ Lecturer	Internal Examiner/ Lecturer
Xavier International College	Xavier International College
……	……
Mr. Rajesh Nepal	External Examiner
HOD, Department of Computer Application	BCA, TU
Xavier International College	

ACKNOWLEDGMENT

We would like to take this opportunity to express our sincere thanks to the Department of Computer Application, Xavier International College for providing us this opportunity to explore our ideas through the project. We are also deeply grateful to **Mr. Rajesh Nepal**, Head of Department for wholehearted support.

It gives us immense pleasure to express our heartfelt sincere appreciation and gratitude to our highly respected and esteemed supervisor **Mr. Santosh Neupane**, for his unwavering commitment and willingness to share his knowledge for this whole work. We would also like to thank **Mr. Rajesh Nepal** for his support and encouragement throughout the project's completion.

We would also like to thank the Computer Application Department for providing us with a conducive environment where we could work on this project. Equally, we are grateful and feel indebted to all our dear friends for their frequent and everlasting support for the completion of this report. Similarly, we extend our sincere thanks to all other seen and unseen personalities involved in the preparation of the project work.

Arjun Raj Acharya (54602068)

Asha Mohan Tamang (54602069)

ABSTRACT

Nepali Sign Language uses distinct hand shapes to represent the 36 consonant characters of the Devanagari script. However, limited automated recognition systems are available for NSL, particularly for real-time, mobile-based applications. This project presents a real-time NSL character recognition system implemented as an Android application that recognizes individual characters, combines them into sequences, and generates spoken Nepali sentences.

The system uses hand landmarks extracted using Media Pipe. The 21 detected hand landmarks are transformed into a 78-dimensional feature vector comprising joint positions, fingertip distances, and finger-bending information. The features are normalized to reduce the effects of hand position, distance from the camera, and variations in the environment, making the system more robust to changes in lighting and background. A lightweight neural network is then used to classify the extracted features into one of the 36 NSL characters.

A dataset containing 7,200 images representing all 36 characters was collected using a purpose-built data-capture application. The trained model achieved 94.4% accuracy on test images. The trained model was converted to TensorFlow Lite and deployed for fully on-device inference, enabling real-time recognition without requiring an internet connection. To improve recognition stability, a character is accepted only after it remains consistent across several consecutive frames. The recognized characters are then combined into sequences and processed by a large language model to generate meaningful Nepali sentences, which are subsequently converted to speech. The system demonstrates the feasibility of combining hand-landmark-based recognition, on-device machine learning, and language processing to support accessible and real-time Nepali Sign Language communication.

Keywords: *Nepali Sign Language, hand landmark detection, Media Pipe, gesture recognition, on-device inference, TensorFlow Lite, neural network*

TABLE OF CONTENTS

ACKNOWLEDGMENT ..... **iv**

ABSTRACT ..... **v**

TABLE OF CONTENTS ..... **vi**

LIST OF ABBREVIATIONS ..... **ix**

LIST OF FIGURES ..... **x**

LIST OF TABLES ..... **xi**

CHAPTER 1: INTRODUCTION **1**

1.1 Introduction 1

1.2 Problem Statement 2

1.3 Objectives 3

1.4. Scope and Limitation 3

1.4.1 Scope 3

1.4.2 Limitations 3

1.5. Development Methodology 4

1.6 Report Organization 6

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW **7**

2.1 Background Study 7

2.1.1 Nepali Sign Language Fingerspelling 7

2.1.2 Computer Vision and Hand Detection 8

2.1.3 Hand Landmarks 9

2.1.4 Image Preprocessing 9

2.1.5 Temporal Stabilization 9

2.1.6 Text-to-Speech 9

2.2 Literature Review 10

CHAPTER 3: SYSTEM ANALYSIS **13**

3.1 System Analysis 13

3.1.1 Requirement Analysis 13
3.1.2 Feasibility Analysis 17
3.1.3 Object-Oriented Analysis 19
CHAPTER 4 : SYSTEM DESIGN 24
4.1 Design 24
4.1.1 System Architecture Design 24
4.1.2 Dataset Design 25
4.1.3 MLP Model Architecture 25
4.1.4 Class Diagram 26
4.1.5 Sequence Diagrams 27
4.1.6 Block Diagram 29
4.1.7 Component Diagram 31
4.2 Algorithm Details 32
4.2.1 Recognition Algorithm 32
4.2.2 Sentence Formation Algorithm 32
4.2.3 Error Handling Algorithm 33
4.2.4 Security Design 33
CHAPTER 5: IMPLEMENTATION AND TESTING 34
5.1 Implementation 34
5.1.1 Tools Used 34
5.2. Testing 36
5.2.1. Test Cases for Unit Testing 36
5.2.2. Test Cases for System Testing 43
5.2.3 Test Results 45
5.3. Result Analysis 47
5.3.1 Media Pipe Hand Detection 47
5.3.2 Multilayer Perceptron 47

CHAPTER 6 : CONCLUSION AND FUTURE RECOMMENDATION 50

6.1. Conclusion 50

6.2 Future Recommendation 51

REFERENCES 52

LIST OF ABBREVIATIONS

API	Application Programming Interface
ASL	American Sign Language
CNN	Convolutional Neural Network
FR	Functional Requirement
IT	Information Technology
MLP	Multilayer Perceptron
NSL	Nepali Sign Language
RNN	Recurrent Neural Network
SDK	Software Development Kit
TFLite	TensorFlow Lite
TTS	Text-to-Speech
UC	Use Case

LIST OF FIGURES

Figure 1: Diagram Representing Project Workflow 5
Figure 2: Nepali Finger Spelling [20] 8
Figure 3: Use Case Diagram 14
Figure 4: Project Schedule 18
Figure 5: State diagram 21
Figure 6: Activity Diagram 23
Figure 7: MLP Architecture 26
Figure 8: Class Diagram of Sign Language Recognition System 27
Figure 9: Sequence Diagram of SignSathi 28
Figure 10: Block diagram of NSL data collector application 29
Figure 11: Block diagram of NSL recognition application 30
Figure 12: Component Diagram 31
Figure 13: Character level NSL recognition 45
Figure 14: Word level NSL recognition 46
Figure 15: Confusion Matrix 48
Figure 16: Model Performance 49

LIST OF TABLES

Table 1: Functional Requirement of the application 13
Table 2: Non-functional requirement 16
Table 3: Sofware Resources and Their Cost 18
Table 4: Recognition Controller Lifecycle 20
Table 5: Process modelling showing full system workflow 22
Table 6: Dataset Collector Application's Camera and Capture Unit Testing 36
Table 7: Dataset Collector Application's Label and Dataset Management Unit Testing 37
Table 8: Dataset Collector Application's Metadata, Settings and Export Unit Testing 38
Table 9: Recognition Application's Camera and Media Pipe Unit Testing 39
Table 10: Recognition Application's MLP Classification Unit Testing 40
Table 11: Recognition Application's Prediction Stabilization and Sentence Processing 41
Table 12: Recognition Application's Text and Speech Unit Testing 42
Table 13: Dataset Collector Application's System Testing 43
Table 14: Recognition Application Application's System Testing 44
Table 15: Overall Model Performance 49

CHAPTER 1: INTRODUCTION

1.1 Introduction

In today's life, communication is important, but there is a need for communication between the Even deaf persons and non-deaf persons who are not familiar with Nepali Sign Language are able to communicate with each other. challenging. Although NSL is widely used in the Deaf community in Nepal, there are not many. The lack of awareness of sign language among the general public and the lack of interpreters. population can create difficulties in places such as healthcare facilities, educational Institutions, offices and public services.

In recent years, advances in computer vision and Artificial Intelligence have made it possible to develop Ways to make sign language recognition easier with cell phones. For example, fingerspelling, in which each Devanagari letter is spelled out. Different hand shapes offer a practical basis for automatic NSL recognition. Fingerspelled characters, as opposed to dynamic signs that require ongoing arm motions, can be recognized from the shape of a hand in an individual frame. tackle for introducing a new way of communicating with the deaf.

The project is developed as a tackle to introduce a new way of communication for the deaf people through SignSathi: Real-Time Nepali Sign Language Recognition. Consonant Recognition System using Mobile Based Assistive Application for 36 Devanagari consonant characters. Recognizes the characters and converts them to words via fingerspelling from a live camera feed. Nepali text to speech. Rather than process the full image of the camera directly, the system uses Media Pipe to detect the user's hand and extract its 21 hand landmarks. Each landmark Has three coordinate values (x, y, z), with 63 landmark features that Show the parts of the hand. These can be interpreted as landmark features and are then preprocessed and fed to A Multilayer Perceptron-classifier for character recognition.

Since predictions are generated continuously from the camera stream, the application also includes a temporal stabilization and buffering mechanism to prevent unstable frame-by-frame predictions from producing repeated or incorrect characters. Confirmed characters are combined into words and utterances based on signing pauses, after which the system

can generate a more natural Nepali sentence and provide spoken output using text-to-speech.

1.2 Problem Statement

The deaf community in Nepal can face difficulties communicating with people who are not familiar with Nepali Sign Language. The limited availability of NSL interpreters can make communication more challenging in everyday situations such as hospitals, educational institutions, government offices, workplaces, and other public settings. Depending on an interpreter for routine communication may not always be convenient, affordable, or immediately possible.

While the computer vision and artificial intelligence field have gained momentum towards the possibility of recognizing sign language, limited resources and practical solutions are available for Nepali Sign Language. Furthermore, a model can be trained on hand images which are more clearly presented but may not be successful when directly applied to a mobile camera where the hand can be located in various positions, sizes, backgrounds and lighting. Therefore, the system should detect the hand and separate it from the background scene in order to recognize the sign.

One difficulty is that a live camera continually generates predictions and if the user has a single sign in his hand, then the character can be repeated or fluctuating. If all of the frame level predictions were simply to be presented, it would make it hard to read the resulting text. The system has to be capable of stabilizing predictions, accepting a character only when his or her hand is kept in an almost fixed position, and inferring the end and beginning of words and utterances from pauses in signing.

This project aims at overcoming these challenges by providing a sign language fingerspelling recognition system in the Nepali language in real-time and mobile based recognition system. This is done using Media Pipe's hand detection and landmark extraction to get 63 three-dimensional coordinate features from the hand. These significant features are passed on to a custom Multilayer Perceptron Classifier which is used to recognize 36 Devanagari consonant characters. Recognized characters are then fixed and made into words and sentences that can then be translated to speech in Nepali language.

1.3 Objectives

i. To develop a real-time Nepali Sign Language recognition system capable of identifying Nepali Sign Language characters from live camera input.
ii. To design and develop a user-friendly mobile application that processes hand signs and converts the recognized characters into Nepali text and spoken output.
iii. To assess the effectiveness and reliability of the developed system through appropriate performance evaluation and testing.

1.4. Scope and Limitation

1.4.1 Scope

This project focuses on developing a real-time Nepali Sign Language recognition system as a mobile-based assistive application. The system is designed to recognize 36 Devanagari consonant characters used in NSL fingerspelling from live camera input. It uses hand detection and image processing to identify the relevant hand region before classifying the sign. The recognized characters are processed to form words and sentences, which can be displayed as Nepali text and converted into speech. The system is primarily intended to support communication between NSL users and people who may not be familiar with sign language, while also serving as a learning aid for NSL..

1.4.2 Limitations

The project works well, but there are some limitations:

i. The system is limited to single-handed signs and does not currently support two-handed signs at the same time.
ii. The recognition is limited to static fingerspelling characters and does not support dynamic signs that require hand movement over time.
iii. The current system recognizes only 36 Devanagari consonant characters and does not including vowels, numerals, or word-level signs.
iv. Recognition performance may be affected by camera conditions, such as poor lighting, unsuitable framing, or difficulty detecting the hand in complex backgrounds.

1.5. Development Methodology

This project's development is through a mix of Waterfall and Iterative development approaches. The general outline of the project is chronological, because There are some activities that require certain input, such as requirements analysis, preparation of the dataset, and designing its system.to be finished prior to implementation. Meanwhile, development, testing, and modification of the models. Iterative development is necessary for application integration as a result of the outcome achieved in each iteration. development

i. **Requirement Analysis**

To set up a system, the requirements need to be identified by stating the objectives, target users, and target system to be developed. Functional needs, and constraints of Nepali Sign Language recognition application.

ii. **Data Collection and Preprocessing**

The data were collected and preprocess data Collection and Preprocessing. Character images are gathered and grouped in a structured dataset of NSL. A dedicated dataset collection application is used to detect the hand, crop the relevant part of the image using the same application. Using the same application, the relevant part of the picture is cropped by using the hand detection. zone and ensure uniformity of image format. Data collected is then compiled and preprocessed for model training.

iii. **Designing a system**

The overall system architecture is designed, mobile applications, handsets, and the cloud system, detection and preprocessing pipeline, recognition model, sentence processing, and speech output. The machine learning model is also created based on the requirements for a lightweight mobile based recognition system.

iv. **Model Construction and Training**

Various stages in the development of the model are performed to get an accurate and efficient recognition model. It's evolved from a bigger baseline model via Ensemble learning & Knowledge distillation to a lightweight MobileNetV3-Small Model appropriate for mobile use.

v. **Mobile Application Implementation**

The trained model is used for recognition in the Flutter mobile app. The implementation involves live camera input, hand localization, image preprocessing, and more. Character Recognition, Prediction Stabilization, Text Formation, and Speech Output.

vi. **Testing and Evaluation**

The system is tested to ensure the proper operation and reliability of the critical components, Pre-processing, Character stabilization, Sentence buffering, Model integration, and mobile application actions. The recognition system is also evaluated using an appropriate model and system performance measures.

vii. **Modification and Improvement**

Modifications are made, as appropriate, to make improvements based on test and evaluation. The recognition performance, stability, usability, and overall system reliability. This iterative process continues until the system meets the intended project requirements.

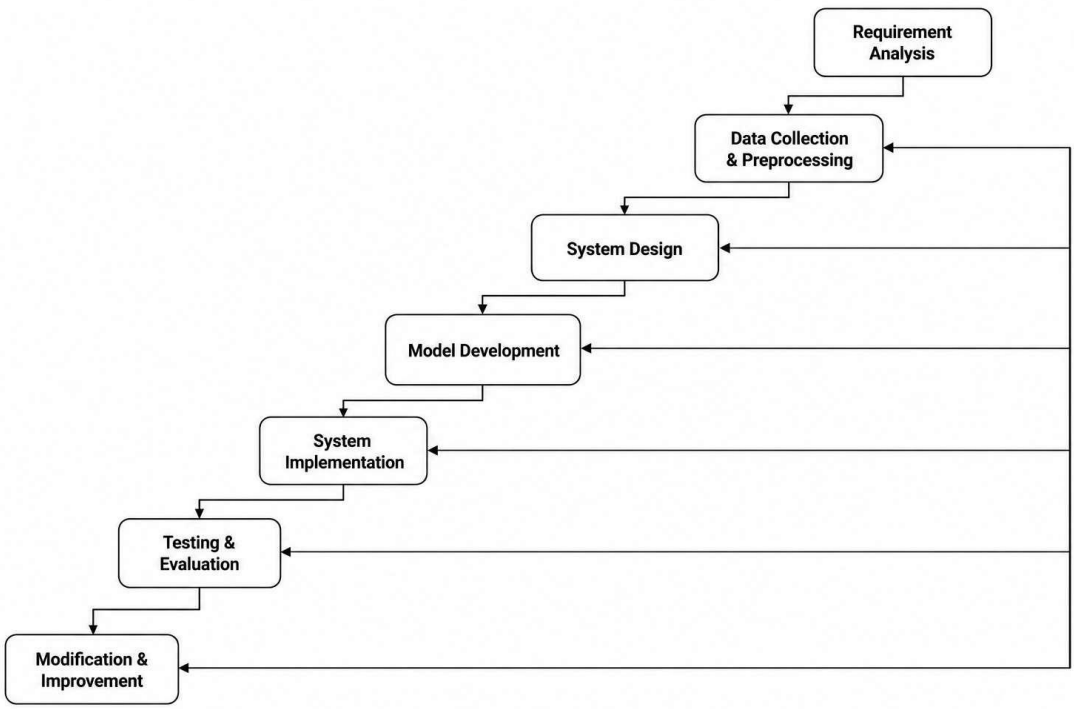


Figure 1: Diagram Representing Project Workflow

1.6 Report Organization

The Chapter 1 gives an introduction to the project and an overview of recognition system of Nepali Sign Language. It includes introduction, problem statement, objective, scope and limitation and development methodology used in the project.

The background study and literature review are provided in Chapter 2. describes sign language recognition, hand detection, image processing and machine learning concepts and technologies. It also carries out a review of the relevant research papers and existing systems.

Chapter 3 deals with system analysis, feasibility study of the proposed system and functional and non-functional requirements of the system. It outlines the needs for the recognition system and mobile app.

The system design is described in Chapter 4. It introduces the overall system architecture, dataset design, hand landmark detection and preprocessing based on Media Pipe, character classification based on MLP, stabilization and formation mechanism of prediction and the interface design of the mobile application.

The implementation, testing and evaluation of the system are covered in chapter 5. It explains the process of dataset collection, hand detection using Media Pipe, extracting and pre-processing the landmarks, character recognition using MLP, stabilizing the prediction, forming sentences, and speech output. It further presents the testing procedure and the results of the evaluation.

The conclusion and future recommendations are given in Chapter 6. It outlines the work accomplished, covers the general conclusions of the project and explores potential improvements and extensions that might be incorporated in future development.

CHAPTER 2: BACKGROUND STUDY AND LITERATURE REVIEW

2.1 Background Study

Nepali Sign Language is used by the deaf community in Nepal for communication. In NSL fingerspelling, individual Devanagari characters are represented using specific hand shapes. Since these characters are represented as static hand configurations, computer vision and image classification techniques can be used to recognize them from camera images.

Recent developments in computer vision and deep learning have made it possible to develop sign language recognition systems that can operate on mobile devices. The problem is that the hand sign recognition from a live camera is not the same as the hand sign recognition of a well-cropped dataset image. In order to recognize the hand shape, the system has to find the hand position and eliminate irrelevant background data, preprocess the image, and finally classify the hand shape. Moreover, there's repetitive input of camera frames which require them to be stabilized in time to create meaningful characters and sentences.

The system therefore combines hand detection, image preprocessing, deep learning-based classification, temporal processing, and speech synthesis to provide an end-to-end Nepali Sign Language recognition system.

2.1.1 Nepali Sign Language Fingerspelling

Fingerspelling in Nepali Sign Language is the representation of the Devanagari characters through particular static handshapes. Fingerspelling can be helpful in signing names, terms and phrases, and concepts for which there are no dedicated lexical signs. Since the handshape for each character is relatively static, fingerspelling can be considered an image classification problem. The proposed system will be based on 36 Devanagari consonant characters but will not be capable of recognizing dynamic signs, vowels, numerals, and word-level signs.

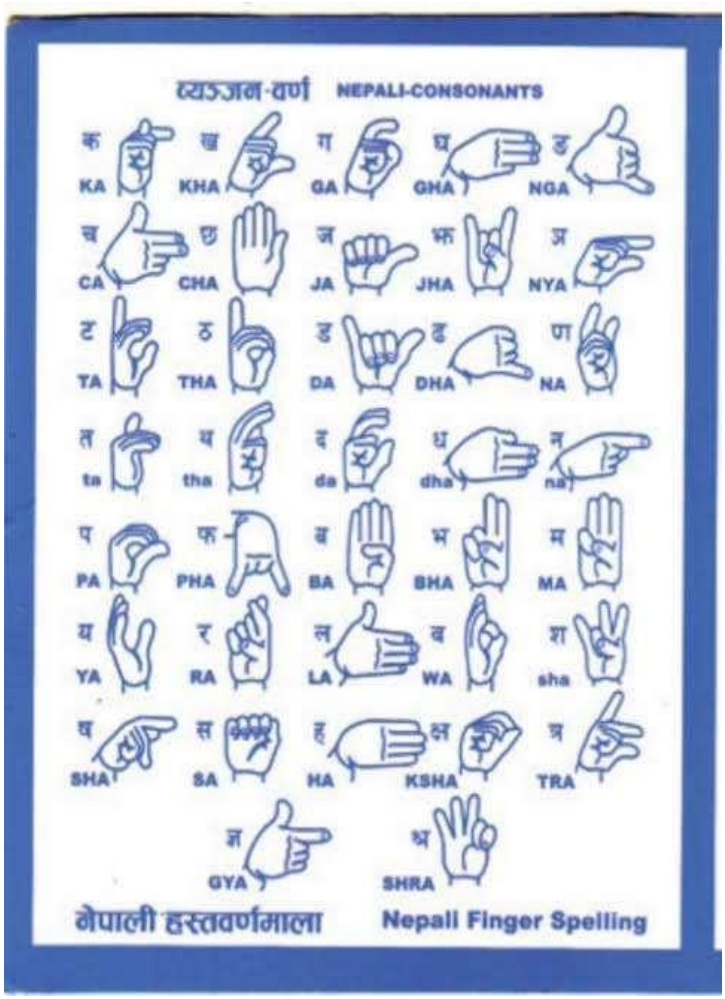


Figure 2: Nepali Finger Spelling [20]

2.1.2 Computer Vision and Hand Detection

Computer vision technology is the ability of a computer to extract useful information from images and videos. In a sign language recognition system, one of the critical tasks is to recognize the presence of a hand within the camera frame before recognizing its shape. Processing the whole camera frame can bring in some unnecessary information and decrease recognition performance if the hand occupies only a small portion of the frame.

The system uses Media Pipe palm detection to detect the user's hand. This detection is used to generate focused input for the recognition model. Media Pipe provides hand localization information that is needed to separate the relevant hand region from the rest of the camera scene.

2.1.3 Hand Landmarks

Hand landmarks are key reference points used to describe the structure and location of a hand. Media Pipe Hand land marker detects 21 landmarks on a hand, each having three-dimensional position information in the form of x, y, and z coordinates. So, a detected hand is described by 63 coordinate values (21 × 3).

In the system, hand landmarks will be used as the main input features for character recognition. These coordinates are extracted from the detected hand and processed into the numerical form for input into the classification model. Using hand landmarks allows representing the hand structure without processing the whole camera frame.

2.1.4 Image Preprocessing

Image preprocessing refers to the process of converting the raw camera input data into an appropriate format before any processing is carried out. In the system design, image preprocessing is mainly done to the hand landmark points.

Once the Media Pipe algorithm detects the hand, 21 points are captured, along with their corresponding x, y, and z coordinate positions. The 63 coordinate points are further processed and normalized in order to reduce variability in the coordinates due to changes in hand position, size, and orientation.

2.1.5 Temporal Stabilization

In a real-time recognition system, the predictions are continuously being made from the frames of the camera. With each frame prediction shown directly as a character, there could be repetition of the same character and wrong predictions may also occur during the process of changing from one sign to another by the user.

Stabilization in time is thus required to make sure that the prediction is stable enough to be accepted. In this system, the character will only be confirmed once consistent predictions are made across multiple frames. This will help reduce the effect of flickering predictions as well as ensure that no unnecessary characters are added to the output due to wrong predictions.

2.1.6 Text-to-Speech

TTS is a technology that converts written text into spoken audio. It allows a computer or mobile application to produce speech from generated text.

In the proposed system, the recognized characters are combined to form Nepali words and sentences. The resulting Nepali text can then be passed to the text-to-speech component so that the recognized message can be spoken aloud. This provides an additional communication method for users who may not understand sign language.

2.2 Literature Review

Sign language recognition has received much attention due to the progress of computer vision and deep learning methods. Previous methods were primarily based on handcrafted features and handcrafted conventional machine learning techniques, while recent research has been increasingly using convolutional neural networks, recurrent neural networks, Transformers, and pose-based representations. Rastgoo et al.[1] gave a thorough overview of sign language recognition and drew attention to the fact that signer variability, illumination, occlusion, background complexity, co-articulation, and availability of training data are all factors that influence sign language recognition performance. Likewise, Alam et al. [2] performed a systematic review of sign language recognition on smart phones and found that many studies claimed high recognition accuracy but relatively few of them considered computational cost, memory requirement, latency and actual performance on smart phones. This means that having high classification accuracy is not enough if the goal is to deploy this target application into the real-world mobile deployments.

Sign language data sets have also played a major role in the advancement of Deeps Learning based sign language recognition systems. Joze and Koller [3] collected a large-scale American Sign Language (ASL) dataset called MS-ASL that features videos of a wide range of signers captured under uncontrolled conditions. They found that signer independent evaluation was important and exhibited differences in recognition performance as a function of vocabulary size and training data. Li et al. [4] created WLASL dataset for word level recognition of ASL and tested various recognition approaches with different sized vocabularies. Importantly, their study showed that structured information like pose or key points can be useful discriminative information in addition to raw RGB video. Lab collected datasets were also shown to be limited by Desai et al. [5] who introduced a community sourced dataset (ASL Citizen) with recordings in diverse real-world environments. On their results, they found that there was a significant drop in performance when tested with signer-independent and real-world data sets, which shows the importance of studying sign language systems beyond the controlled data sets.

When it comes to real-time sign language recognition, hand landmark extraction offers an appealing alternative to considering full video frames. A novel framework for hand detection and tracking was proposed by Zhang et al. [6], which is named Media Pipe Hands and is designed to be executed on device to detect and track 21 3D landmarks of each hand. The structure includes a palm detection stage, landmark estimation and tracking stage, which is suitable for real-time applications on mobile devices. De Coster et al. [7] examined robust sign embeddings for low-resource sign languages with key point-based sign representations, demonstrated that using the appropriate landmark normalization and embedding can enhance recognition and transferability between sign languages. More recently, Zhang and Notni [8] showed that 3D geometric features combined with Media Pipe hand skeletons is an effective approach for recognizing ASL that learned using PointNet and an MLP. Similarly, Uddin et al. [9] used Media Pipe landmarks in conjunction with an LSTM to recognize Norwegian Sign Language (NSL) in real-time, proving that this representation could be used in low-resource sign language environments.

The use of lightweight models is especially crucial for mobile sign language recognition. Rathi[10] studied the transfer learning methods for recognition of sign language on a mobile platform. The best system yielded an average of around 2.42 seconds per recognition on the mobile platform while showing about 95.03 % recognition accuracy. This is also a good example of the trade-off between classification accuracy and practical inference latency. Transferring a high capacity image-based CNN to the mobile application, therefore, may not give sufficient real-time user experience. More compact architecture and landmark-based approaches have therefore been investigated recently. To reduce computation requirements while maintaining sign language recognition, Li et al. [11] presented a lightweight SL recognition method based on Media Pipe and knowledge distillation. Varanasi et al. [12] also studied isolated sign language recognition using the landmarks provided by Media Pipe and demonstrated the potential of compact landmark based models for efficient recognition.

In the context of Nepal, Acharya and Pant[13] investigated computer-vision based hand gesture recognition for persons with speech disability which laid a foundation for computer vision-based gesture recognition in the context of Nepal. To show the applicability of deep learning methods for recognition of NSL signs, Mali et al. [14] used a two-dimensional CNN. Thapa et al. [15] conducted research on finger spelling recognition in Nepali sign

language (NSL) using the machine learning techniques, and also, Ligal and Baral [16] studied deep learning-based gesture recognition of NSL. While these studies confirm that deep learning can be effectively used for NSL, most of the existing methods have been developed to recognize the image with only a focus on accuracy. They lack comprehensive consideration of the real-time, offline, and resource-constrained mobile deployment.

Heuju and Rijal [17] explored the word-level recognition of NSL using Transformer-based approach and found that the recognition rate drops with the increase of vocabulary. For NSL, Shrestha et al. [18] explored two tasks with Media Pipe and CNNs: NSL letter detection and finger spelling, proving the applicability of landmark-based hand representation in NSL. Sunuwar et al. [19] introduced "NSL23" dataset for Nepali Sign Language alphabets which is an important dataset resource for the further research. Bhusal and Baral [20] furthered NSL recognition by introducing sentence-level recognition; which showed the possibility of recognizing continuous Nepali Sign Language. To further explore NSL character recognition, Poudel et al. [21] also developed data sets and deep learning methods, highlighting the need for high-quality data sets and selecting appropriate deep learning methods for NSL recognition.

Although these advancements have been made, there is an important research gap in the development of lightweight, real time, offline Nepali Sign Language (NSL) recognition system applicable to mobile devices. Current NSL research primarily focuses on recognition accuracy, leaving the other factors such as low latency, small model size, offline inference, signer variations, and continuous recognition inadequately explored. The literature also suggests that the problem of low-resource SLs is not necessarily solved by making a model more complex, as the size of the vocabulary, the number of different type of data sets, and the type of variation among Signers are still significant limitations [3], [5], [17]. Thus, a novel first-of-its-kind method based on Media Pipe for hand feature extraction and a lightweight deep learning model is promising. This can decrease the amount of calculations required to process raw frames of video and still preserve the spatial information of the hand configurations.

CHAPTER 3: SYSTEM ANALYSIS

3.1 System Analysis

3.1.1 Requirement Analysis

Requirement Engineering or Requirement Analysis refers to the process of identifying the needs and expectations of a product or system. It helps to ensure that the software meets its desired goal and specifications. Requirements are split into two main types: functional requirements and non-functional requirements. The process of requirement analysis ensures that the requirements are clear, unambiguous, measurable and well documented.

3.1.1.1 Functional requirements:

The functional requirements describe what a system should do and how it should interact with users. They describe the functionalities of the system. The project's functional requirements are as follows:

Table 1: Functional Requirement of the application

ID	Requirement	Description
FR-1	Recognition System Initialization	The system shall initialize the recognition components, including the recognition controller, Media Pipe hand detection, and the deployed character classification model before starting recognition.
FR-2	Camera Initialization	The system should access and initialize the mobile device camera for sign recognition.
FR-3	Real-Time Sign Capture	The system shall continuously capture frames from the camera feed for real-time sign recognition.
FR-4	Hand Detection and Landmark Extraction	The system will detect and localize the user's hand from each camera frame using Media Pipe and extract the 21 hand landmarks.
FR-5	Real-Time Sign Recognition	The system shall recognize Nepali Sign Language characters locally using the custom MLP classifier based on the extracted 63 hand landmark coordinate features.

FR-6	Prediction Stabilization and Display	The system shall stabilize consecutive frame predictions and display the recognized characters and confidence information in the application interface.
FR-7	Sentence Formation	The system shall combine recognized characters into words and sentences based on the user's signing sequence and pauses.
FR-8	Text-to-Speech Output	The system shall convert the generated Nepali text into speech when requested by the user.
FR-9	Session Control	The system shall allow the user to start and stop the recognition process during a session.
FR-10	Error Handling	The system shall notify the user when camera access, hand detection, recognition, or other required services encounter an error.

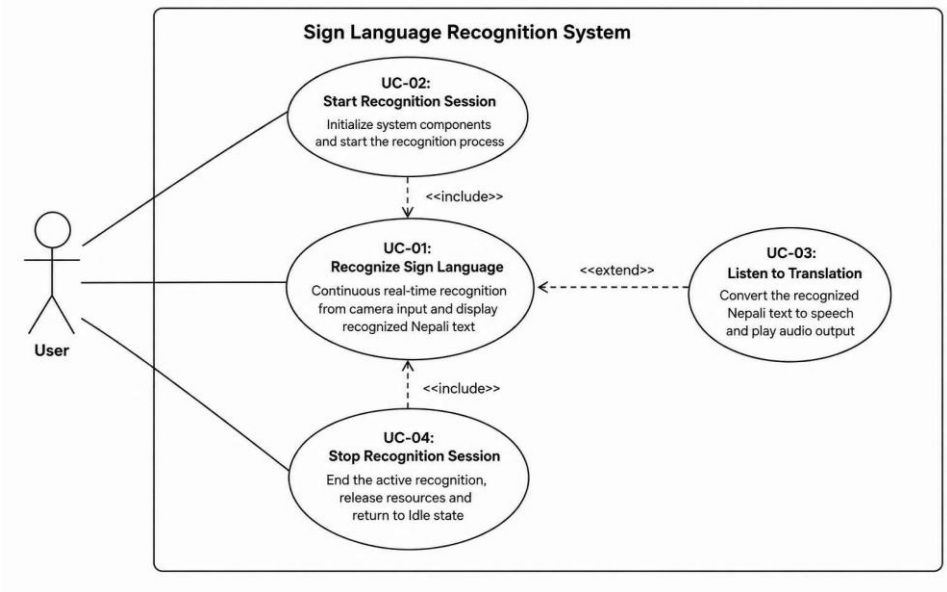


Figure 3: Use Case Diagram

Use Cases Descriptions:

The following four use cases define the primary interactions between the user and the system.

UC-01: Recognize Sign Language

Actor: User

Precondition: Application is installed, camera permission has been granted, and the on-device recognition model is loaded.

Main Flow:

1. The user opens the application and starts a recognition session.
2. The system starts with the Camera Manager and begins capturing frames.
3. Each captured frame is passed through the Media Pipe hand detection and silhouette preprocessing pipeline.
4. The processed frame is passed to the MLP model, which returns a predicted character and confidence score.
5. The stabilization mechanism confirms a character only when consecutive frames agree.
6. Confirmed characters are combined into words based on signing pauses.
7. The recognized Nepali text is displayed on screen in real time.
8. Recognition continues in a loop until the user stops the session.

Postcondition: Recognized Nepali text is displayed on screen in real time.

UC-02: Start Recognition Session

Actor: User

Precondition: Application is open and camera permission has been granted.

Main Flow:

1. The user triggers the start action from the interface.
2. Recognition Controller initializes the Camera Manager and loads the on-device model.
3. The system transitions from Idle state to an active recognition state.

Postcondition: The system is actively capturing frames and recognizing sign language input.

UC-03: Listen to Translation (Text-to-Speech)

Actor: User

Precondition: A recognition result (word or sentence) has been generated and is displayed.

Main Flow:

1. The user requests text-to-speech playback.
2. The system retrieves the latest recognized Nepali text.
3. The system converts the recognized text into speech output.
4. Speech is played through the device speaker.

Postcondition: The recognized text is spoken aloud through the device.

UC-04: Stop Recognition Session

Actor: User

Precondition: A recognition session is currently active.

Main Flow:

1. The user triggers the stop action.
2. The Recognition Controller sends a stop signal.
3. Camera Manager releases the camera resource.
4. Active recognizer is deactivated.
5. The system returns to the Idle state.

Postcondition: The recognition session is terminated and all resources are released.

3.1.1.2 Non-Functional requirements:

Non-functional requirements are those features of the system which do not affect the working functionality of the system in terms of the result but helps to make it more effective and efficient in terms of time, reliability and security. Non-functional requirements are as follows:

Table 2: Non-functional requirement

Category	Attribute	Requirement
Performance	Latency	Recognition latency shall remain below one second.

Performance	Frame Processing	The application shall process camera frames efficiently without noticeable lag.
Reliability	Stability	The application shall maintain stable operation during continuous recognition sessions.
Reliability	Recovery	Recognition services shall recover gracefully from temporary failures.
Usability	Interface	The user interface shall remain simple and intuitive, requiring minimal interaction to initiate recognition.
Security	Data privacy	The application shall perform sign language recognition entirely on device.
Security	Data Storage	User configuration data shall be stored securely on device.
Portability	Platform	The application shall support Android devices running Android 8.0 (API level 26) and above.

3.1.2 Feasibility Analysis

The feasibility analysis of a project is carried out to determine whether a project can be completed within the constraints of time, budget, technology and other resources.

i. Technical Feasibility

The system is technically feasible as it can be developed by using the technologies which are easily available in the market and are established in the market. Flutter provides a good platform to build the mobile app from a common codebase, and TensorFlow Lite makes it easy to do model inference on device on mobile devices. Media Pipe is used for reliable landmark extraction and detection during the recognition process. The recognition model is lightweight and is engineered to run in the resource-constrained environment of a mobile device. The technologies chosen are supported and can be applied to the scope and timeline of the project to accommodate the required real-time recognition, text processing, and text-to-speech functions.

ii. Economic Feasibility

The development environment is based entirely on open-source tools and frameworks due to which the cost of licensing is either low or none. The system can be developed and tested using existing computing and mobile resources.

Table 3: Sofware Resources and Their Cost

Resource	Cost
Flutter SDK	Free (open-source)
TensorFlow Lite	Free (open-source)
Media Pipe	Free (open-source)
Python	Free (open-source)
Android Studio	Free

iii. Operational Feasibility

The application is a solution to a real-life communication challenge for Nepali Sign Language users. It needs only a smartphone equipped with a camera – no special equipment is necessary, the system is simple and accessible. The application is intended to conduct the recognition process directly on the mobile phone, with no need for internet connection. This means that the system can be used in various settings everyday.

iv. Schedule Feasibility

The project is scheduled to be completed within the following timeframe:

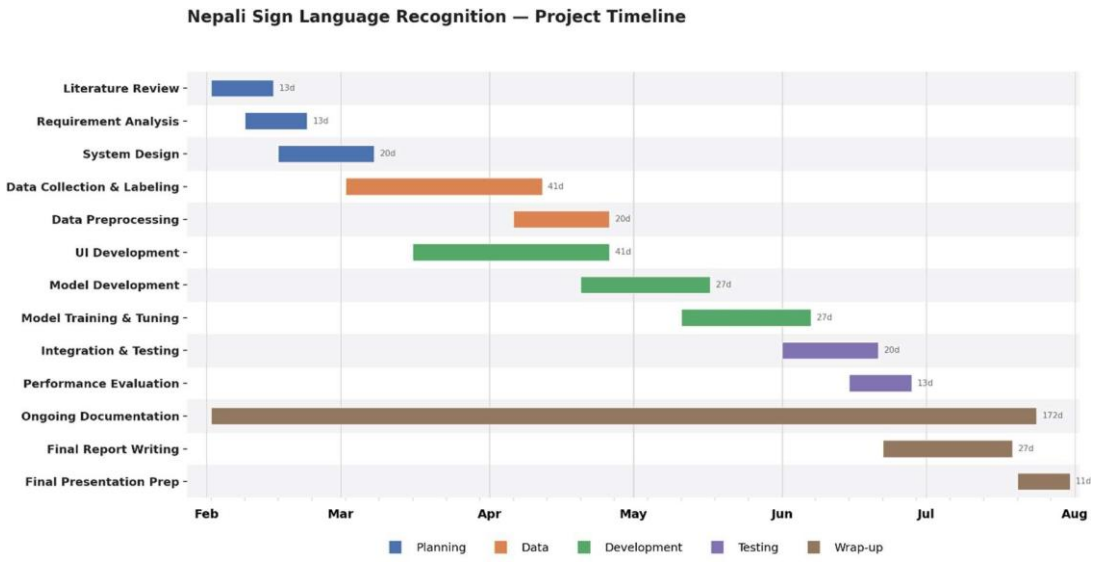


Figure 4: Project Schedule

3.1.3 Object-Oriented Analysis

OOA is the stage at which the system requirements are specified and real-world objects, attributes, behaviors and relationships are determined with respect to the user's point of view.

3.1.3.1 Object Modeling Class Diagram

The Class diagram describes the static structure of the system. Based on functional requirement and use case analysis, four main classes of functions have been identified. The classes and responsibilities identified are listed below:

1. User

It's the end user who is using the system via the application. This is an actor class that contains no business logic; it starts and ends recognition sessions, issues text-to-speech playback requests.

2. Camera Manager

It handles all camera related operations in the application. This will initialize and configure the camera, acquire frames continuously, pass the acquired frames to the Recognition Controller, and free up camera resources at the end of the session.

3. Recognition Controller

It acts as the main orchestrator of the recognition workflow. Handles the full session life cycle (start, run, stop), gets frame stream from Camera Manager and sends out frames to Recognizer. The controller fetches the prediction results, formulates sentences using pauses in the sign, and passes the recognized text to the UI to display.

4. Recognizer

It recognizes hand signs on device by using Media Pipe hand detection, landmark extraction and the custom MLP classification model. It is able to identify the hand in each frame, extract the 21 hand landmarks and acquire 63 coordinate features (x, y, z). The features are preprocessed and given to the MLP classifier for character prediction. The recognizer also performs temporal stabilization of the successive predictions and returns the recognized prediction to the Recognition Controller.

The relationships between the classes are explained as below:

1. The User triggers the recognition process, can pause and resume recognition sessions and can ask for TTS output.
2. A camera is initialized, acquired and released by the Recognition Controller through Camera Manager.
3. The Recognition Controller passes the frames it captures to the Recognizer and gets character predictions and confidence values back.
4. The Recognizer detects the hand using Media Pipe, and extracts 21 landmarks from the hand, after that it uses the MLP classifier to recognize the character.
5. The Recognition Controller stabilizes predictions and groups recognized characters into words and sentences using signing pauses.
6. Its known Nepali text is shown in the application's UI and can be sent to the TTS component, if requested by user.

3.1.3.2 Dynamic Modeling State Diagram

The state diagram models the complete lifecycle of the Recognition Controller, representing how the system transitions between operational states in response to events.

Table 4: Recognition Controller Lifecycle

State	Trigger / Event	Description
Idle	App launched	Application is open and ready to start a recognition session; camera and recognition process are inactive.
Initializing	startSession()	The system initializes the camera, MediaPipe hand detection, and MLP recognition components.
Running	Initialization completed successfully	Camera frames are continuously captured, the hand is detected and landmarks are extracted, and the MLP processes the landmark features to generate predictions.
Stopped	stopSession()	The recognition session is terminated, the camera is released, and active resources are cleaned up.
Idle	restart()	The system returns to the initial state and is ready for a new recognition session.

Within the Running State, the system operates in a continuous internal loop: capture frame, send to recognizer, receive prediction, update display. This loop persists until a stop event is triggered.

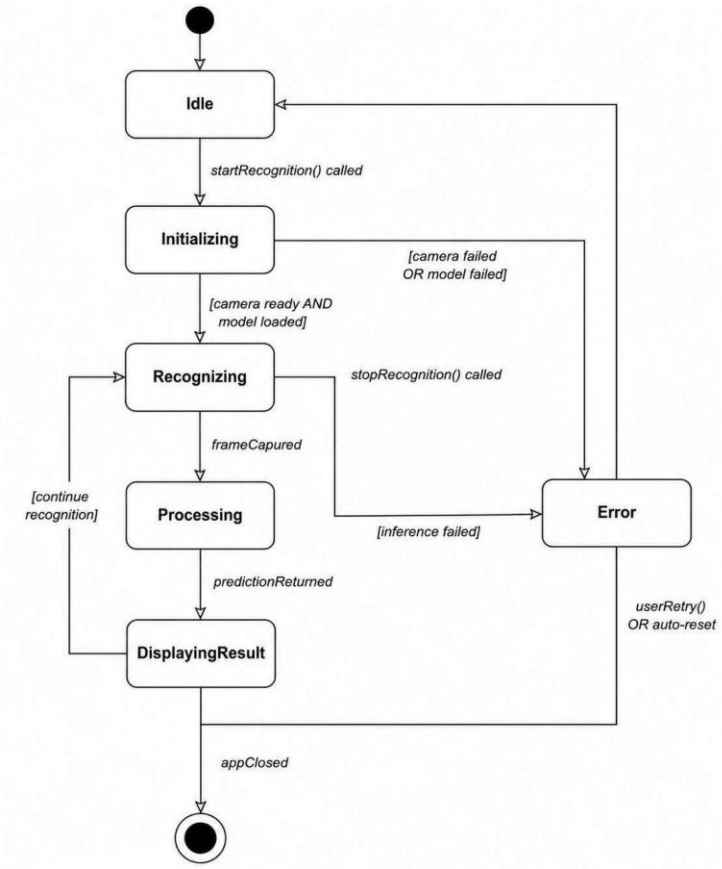


Figure 5: State diagram

3.1.3.4 Process Modeling using Activity Diagram

The activity diagram represents the complete workflow of the application, from launching the application and starting a recognition session to continuously processing camera frames and displaying the recognized output until the user stops the session.

Table 5: Process modelling showing full system workflow

Step	Activity
1	**Start:** User opens the application.
2	The user starts a recognition session from the application interface.
3	The system initializes the Recognition Controller and Camera Manager and loads the on-device MLP recognition model.
4	The camera begins capturing frames.
5	**LOOP:** A camera frame is captured.
6	**LOOP:** The captured frame is processed using Media Pipe hand detection, and the 21 hand landmarks are extracted.
7	**LOOP:** The extracted 21 hand landmarks (63 x, y, and z coordinate features) are preprocessed and provided to the MLP classification model.
8	**LOOP:** The system receives the predicted character and confidence score from the MLP classifier.
9	**LOOP:** The stabilization mechanism processes consecutive predictions to confirm the recognized character.
10	**LOOP:** Confirmed characters are combined into words and sentences based on signing pauses.
11	**LOOP:** The recognized Nepali text is displayed on the application interface.
12	**Optional:** The user requests text-to-speech, and the recognized Nepali text is converted into spoken output.
13	**Decision:** Continue recognition → return to Step 5. Stop recognition → proceed to Step 14.
14	The system stops recognition and releases the camera and other active resources.
15	**End:** The recognition session is terminated.

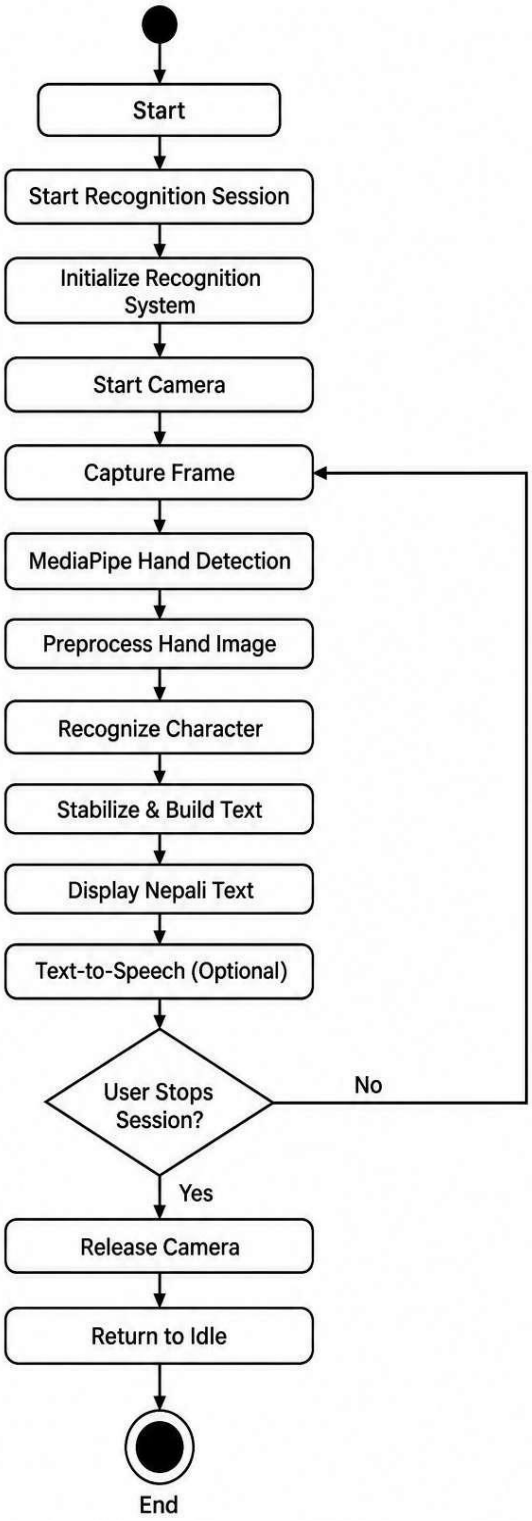


Figure 6: Activity Diagram

CHAPTER 4 : SYSTEM DESIGN

4.1 Design

The system design of the Sign Language Recognition System is based on Object-Oriented approach. The design phase turns the analysis artifacts into actual, executable design specifications including class diagrams, interaction diagrams, state machines, component structures, and deployment topology. These diagrams have been reviewed and refined to accurately represent the architecture of the system for on-device recognition with a focus on edges.

4.1.1 System Architecture Design

Notably, the Sign Language Recognition System is edge-based with the complete recognition system operating on the mobile device. Hand detection and landmark extraction is performed using Media Pipe; extracted landmark coordinates are used as input for an MLP model for sign language character classification without any network connection and without a server-side component.

The architecture is organized into the following major subsystems:

1. User Interface Layer: Manages the interactions of the user, begins and ends the recognition session and presents the recognized Nepali text.
2. Camera Acquisition Layer: Grabs the video frames of the device's camera in real time and supplies them to the recognition pipeline.
3. Recognition Controller: Controls recognition session and coordinates camera frame flow, hand detection, landmark extraction, hand classification, and output generation.
4. Hand Detection and Landmark Extraction Module: Detects the hand in each camera frame using Media Pipe and extracts 21 hand Landmarks, each landmark has its own X, Y, and Z coordinates. This produces 63 coordinate values that represent the hand.
5. Recognition Engine: Accepts the extracted landmark coordinates and utilizes the MLP model to recognize the performed Nepali Sign Language character and make prediction.
6. Prediction Stabilization and Text Formation Module: Takes care to stabilize the prediction output at the frame level and merges the characters that have been confirmed into words and sentences.

7. Text-to-Speech Module: Transforms the Nepali output into speech output as per user demand.
8. Configuration Management Module: Allows us to manage application configuration and recognition related configuration for the system.

4.1.2 Dataset Design

The recognition model is trained on a custom Nepali Sign Language fingerspelling dataset built specifically for this project using a purpose-built data-capture application. The dataset contains 7,200 images spanning all 36 Devanagari consonant character classes used in NSL fingerspelling.

For each captured image, Media Pipe Hands is used to detect the hand and extract 21 hand landmarks, each with x, y, and z coordinates, giving 63 raw coordinate values per sample. These coordinates are further processed into a 78-dimensional feature vector that incorporates joint positions, fingertip distances, and finger-bending information, rather than being used as raw silhouette images. The features are normalized to reduce the effect of hand position, distance from the camera, and orientation, improving robustness to lighting and background variation.

The processed landmark dataset is organized using Pandas, associating each 78-dimensional feature vector with its corresponding character label. The dataset is split into training and test sets using scikit-learn 80/20 prior to model training.

4.1.3 MLP Model Architecture

The classification model is a custom Multilayer Perceptron (MLP) designed to map the 78-dimensional hand feature vector to one of 36 NSL consonant classes. The feature vector incorporates hand joint positions, fingertip distance measurements, and finger-bending information derived from the hand landmarks detected by MediaPipe. These engineered features provide a descriptive representation of the hand configuration for character classification.

The network first applies Gaussian Noise with a standard deviation of 0.05 during training as a regularization technique. The resulting feature vector is then passed through a Dense layer with 256 neurons using ReLU activation, followed by Batch Normalization and Dropout with a rate of 0.3. A second Dense layer with 128 neurons using ReLU activation

is then applied, followed by another Batch Normalization and Dropout with a rate of 0.3. The final output layer contains 36 units, corresponding to the 36 NSL consonant classes, and uses a linear activation to produce the output logits.

The progressively narrowing hidden layer sizes, from 256 to 128 neurons, allow the model to learn increasingly compact representations of the 78-dimensional hand features while maintaining a lightweight architecture suitable for mobile deployment. Batch normalization helps maintain stable activations during training, while dropout helps reduce overfitting.

The model is trained using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy as the loss function. Training is performed for 100 epochs with a batch size of 32. The trained model is subsequently converted to TensorFlow Lite format for lightweight, on-device deployment in the mobile application.

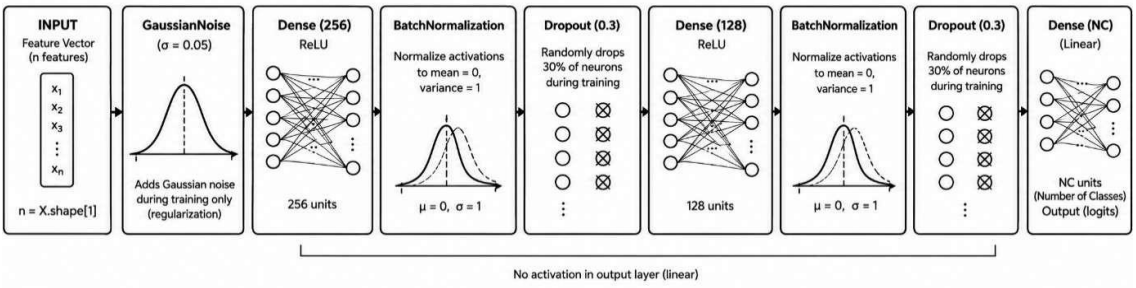


Figure 7: MLP Architecture

4.1.4 Class Diagram

The Class diagram shows the static structure of the application, giving a basic understanding. It shows the principal classes and their connections to one another as well as the attributes and methods of the classes, in particular the User, Recognition Controller, Camera Manager, Recognizer, Sentence Builder, TTS Manager, Prediction classes.

Key design decisions reflected in the class diagram include:

1. The central coordinator is Recognition Controller, which is responsible for coordinating the Camera Manager, the Recognizer, the Sentence Builder and the TTS Manager.
2. Recognizer is responsible for the sign recognition process: detecting hand with Media Pipe, extracting landmarks, passing the extracted landmark coordinates to the MLP model for recognizing the sign and classification of the recognized signs. It returns a Prediction object as the result of the recognition.

3. Sentence Builder confirms predictions and recognizes the characters and builds words and sentences according to timing and pause between characters.
4. The Recognition Controller will call TTS Manager to speak the text being recognized when the user requests it.
5. Camera Manager initializes, captures frames in and releases the camera during the recognition session.

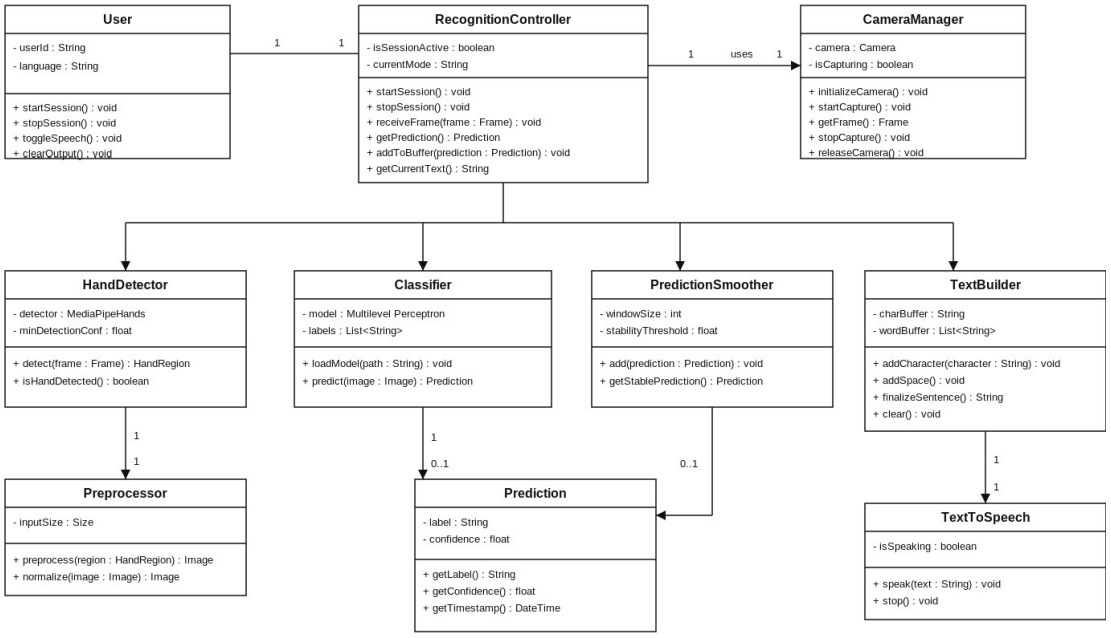


Figure 8: Class Diagram of Sign Language Recognition System

4.1.5 Sequence Diagrams

The sequence diagram shows the time sequence of events among objects for the main recognition scenario, illustrating the complete process from user initiation to the display of results.

The sequence starts with the startRecognition() method, which calls Camera Manager to start the camera and Recognizer to load the trained MLP model (model.tflite). After both are confirmed ready, the system enters a loop: capturing a frame, detecting the hand via Media Pipe and extracting its 21 landmarks, transforming these into a normalized 78-dimensional feature vector, running local TFLite inference via predict(), applying temporal stabilization, and finally displaying a confirmed Prediction result on the User Interface.

When the user requests playback, the Recognition Controller invokes TTS Manager to speak the recognized text. The loop runs until stopRecognition() is called by the user.

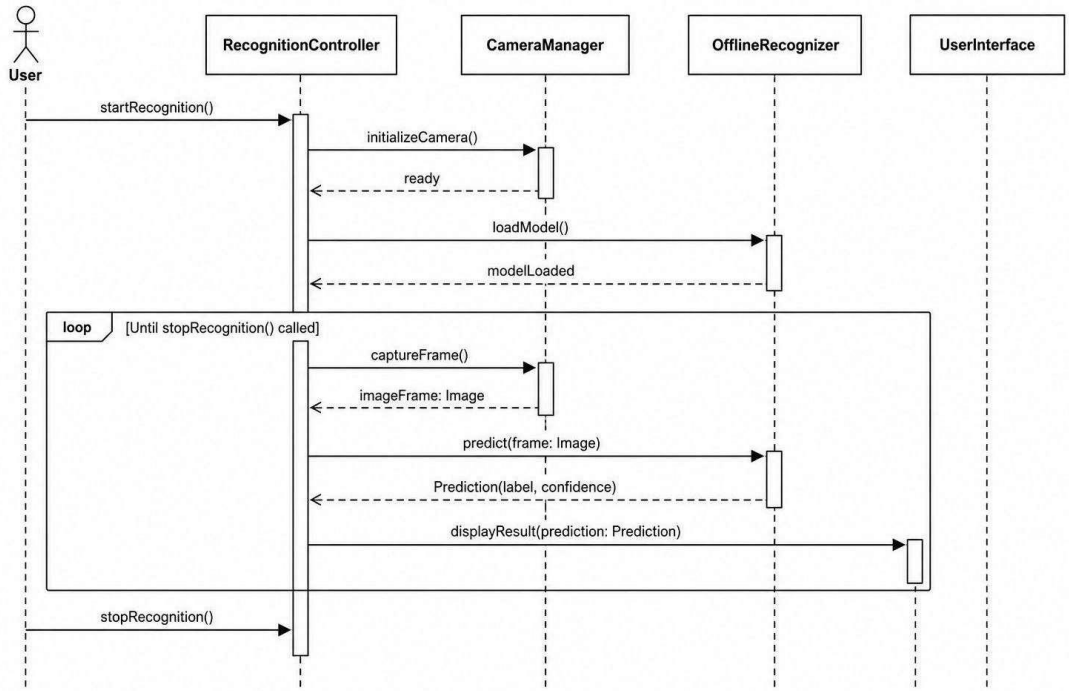


Figure 9: Sequence Diagram of SignSathi

4.1.6 Block Diagram

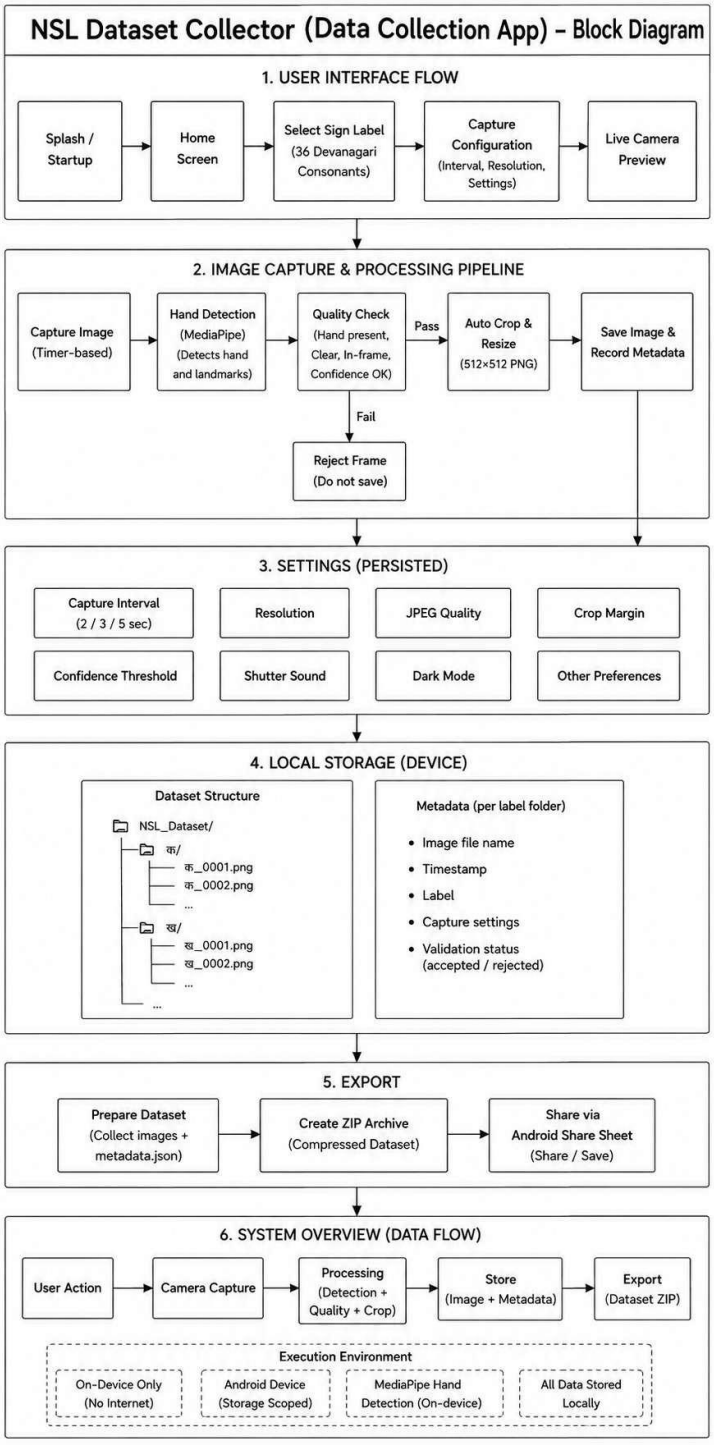


Figure 10: Block diagram of NSL data collector application

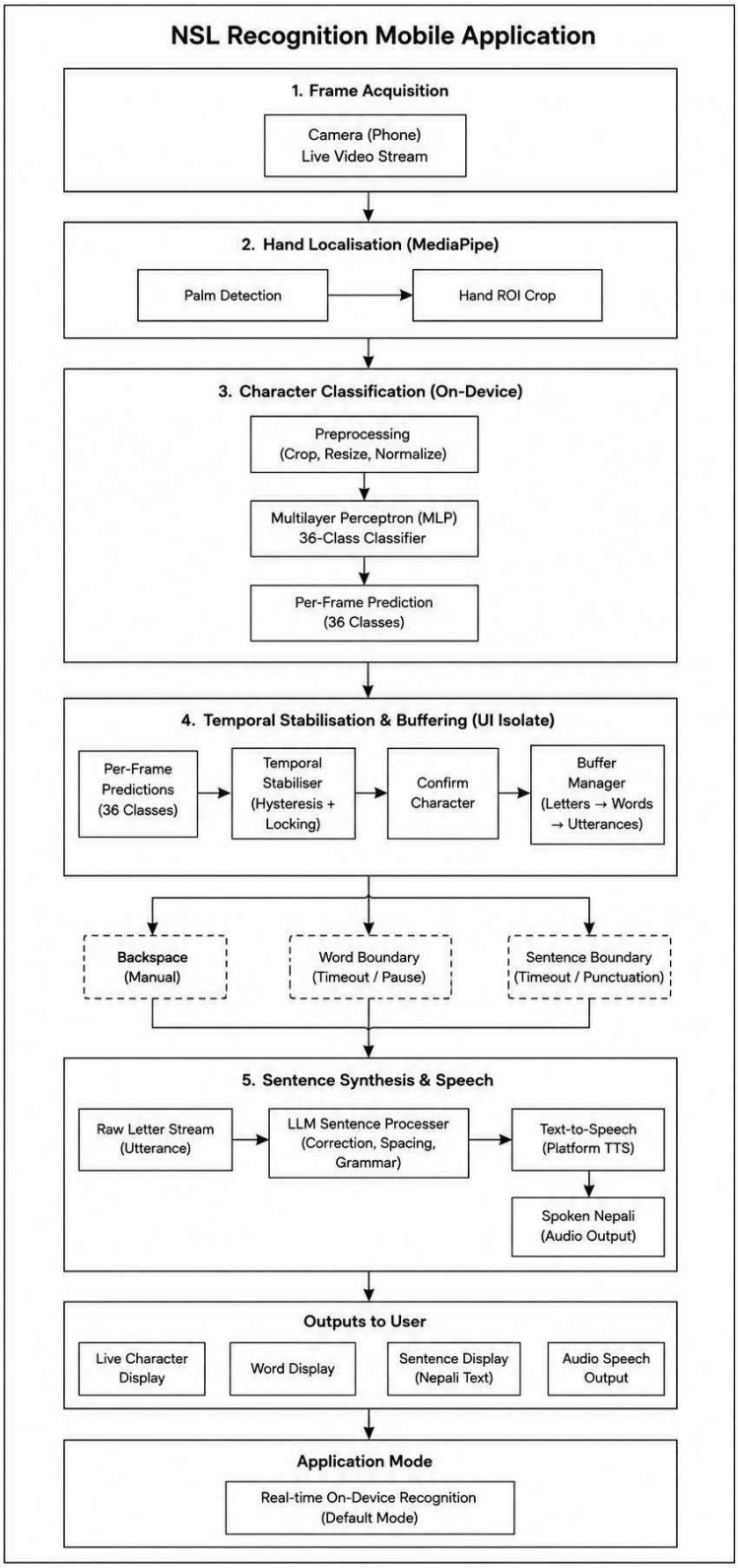


Figure 11: Block diagram of NSL recognition application

4.1.7 Component Diagram

The component diagram depicts the components of the mobile application and the relationships between them. The application consists of the following components: UIComponent, CameraManagerComponent, RecognitionControllerComponent, RecognitionComponent (hand landmark detection via MediaPipe, feature vector preprocessing, and TFLite MLP inference), SentenceBuilderComponent, TTSComponent, and SettingsComponent.

RecognitionControllerComponent serves as the integration point and subscribes to ICameraFrames from CameraManagerComponent, and to IPrediction from RecognitionComponent, as well as to ISettings from SettingsComponent. It exposes IRecognitionResult to UIComponent and passes confirmed characters to SentenceBuilderComponent via ISentenceUpdate. When the generated text is ready to be spoken, UIComponent calls ISpeech on TTSComponent to start speech output. On-device inference requires the trained MLP model file (model.tflite) to be bundled locally, which RecognitionComponent loads and accesses directly.

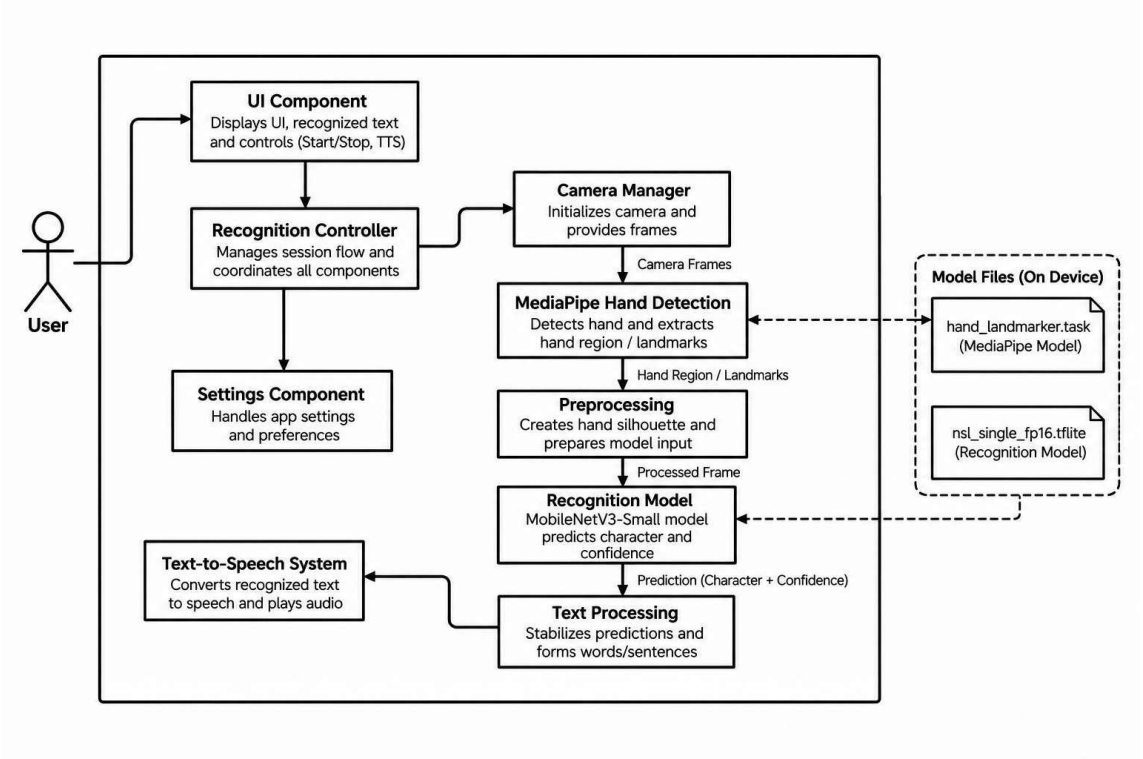


Figure 12: Component Diagram

4.2 Algorithm Details

The following algorithms describe the procedural logic implemented within the Sign Language Recognition System. Separate algorithms are defined for Offline recognition, Online recognition, and error handling.

4.2.1 Recognition Algorithm

The recognition algorithm executes entirely on the mobile device using the TensorFlow Lite model. The algorithm is defined as follows:

1. Initialize the camera module and verify camera availability.
2. Load the trained MLP model, converted to TensorFlow Lite format (model.tflite), for on-device inference.
3. Start frame acquisition from the camera stream.
4. Capture the current image frame.
5. Detect the hand using Media Pipe Hands and extract 21 hand landmarks (x, y, z coordinates).
6. Transform the 63 raw landmark coordinates into a normalized 78-dimensional feature vector (joint positions, fingertip distances, finger-bending information).
7. Run local inference by passing the feature vector through the TF Lite MLP interpreter.
8. Obtain the predicted character label and associated confidence score.
9. Apply temporal stabilization across consecutive frames to confirm the character.
10. Pass the confirmed character to the sentence formation module and display the result on the User Interface.
11. If the recognition session is active, return to Step 4 and repeat.
12. Release camera resources and dispose of the TF Lite interpreter.

4.2.2 Sentence Formation Algorithm

The sentence formation algorithm groups confirmed characters into words and sentences based on timing gaps between signs.

1. Get a confirmed character, along with its recognized timestamp, from the Recognizer.
2. Calculate the difference in time between the last character and the most recent one found.
3. Compare that time difference with the configured Pause Thresholds.

4. If it's a gap at a word boundary, add a word separator; if it's a gap at an utterance boundary, close the current sentence.
5. Add the character to the active word/sentence buffer. 5. Replace the on-screen information in the User Interface. If the recognition session is in progress, go back to Step 1.
6. At session stop, end and return the final sentence (optional) for speech output

4.2.3 Error Handling Algorithm

The error handling algorithm runs concurrently with the primary recognition flow and monitors system health throughout the session.

1. Keep an eye on the availability of camera equipment during the recognition session.
2. Monitor the success rate of hand detection to identify if there are extended periods of time where the hands are not being detected.
3. Identifies any inferences that are failed during the execution of the local prediction.
4. Show a useful error message to the user telling them what went wrong.
5. Give the user a chance to repeat the failed operation or reset to Idle state.

4.2.4 Security Design

The following security measures are incorporated into the system design to ensure reliable and protected operation:

1. On-device processing: All camera frames and recognition data is processed on the device only without being sent over a network which prevents gesture data being exposed to outside interception.
2. Input validation: The input data to the system is validated before use; such as input from the camera frames and configuration values.
3. Secure local storage: Application settings and configuration data is securely stored within the local storage of the application.
4. Permission based camera access: Camera access is requested at runtime, based on a platform permission model, and the user must explicitly grant access.
5. Controlled resource access: Resources like the camera stream and TFLite interpreter are released after the session ends.

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Implementation

Implementation is the phase of the system development where the system designs are converted into working software through coding and programming. Various technologies are used in building software. The tools, technologies and programming languages used for the development of the project are mentioned below.

5.1.1 Tools Used

i. **Python**

The Nepali Sign Language dataset is prepared using python, the hand landmark data is processed and trained the character recognition model and the performance is evaluated by python.

ii. **Media Pipe**

Real-time hand detection and landmark extraction using Media Pipe. It identifies 21 landmarks of the user's hand and provides the x, y, z coordinates for each of these landmarks. These landmarks contain 63 coordinate features that are used for character recognition.

iii. **NumPy**

The numerical operations and processing of the hand landmark coordinates extracted are carried out using NumPy. It helps feature normalization and preparation of the input data to be fed into the recognition model.

iv. **Pandas**

Pandas is used to organize and manage the collected landmark dataset. It assists with the handling of landmark coordinates, character labels and setting up the datasets.

v. **Scikit-learn**

The data is split into training and test sets with scikit-learn and the performance of the recognition model is evaluated using suitable evaluation metrics with scikit-learn.

vi. **Multilayer Perceptron**

The 36 Devanagari consonant characters of Nepali Sign Language fingerspelling are recognized using a custom made MLP classifier. The hand fingerspelling technique of Nepali Sign Language is based on 36 Devanagari consonant hand

characters, which are identified by using a custom made MLP classifier. The preprocessed 63 hand landmark coordinate features are input to the classifier to get the prediction of character and confidence score.

vii. **Flutter**

The mobile application 'SignSathi' has been developed using flutter. It serves as user interface and enables integration with camera, control of recognition session, display of recognized text and interaction with the recognition components.

viii. **Dart**

Dart is used as the programming language for developing the Flutter application. It is responsible for the application logic, user interactions, recognition workflow, text formation, and integration of the text-to-speech functionality.

ix. **Android Studio**

The Android application is developed, run, debugged and tested using the primary development environment—Android Studio. It also contains the Android SDK and tools for deployment and testing the application on the Android device.

x. **OpenCV**

For the dataset collection and preparation parts where necessary, image and camera processing is carried out with OpenCV. It can be used for capturing and managing camera frames, etc.

xi. **Text-to-Speech**

Text-to-speech is used to output the recognized Nepali text. This is another way to communicate when characters have been identified and put together into text.

xii. **Draw.io**

The system architecture, flowcharts, UML diagrams, activity diagrams, class diagrams and other diagrams created for the project documentation are designed using Draw.io.

5.2. Testing

Testing refers to the process of ensuring whether the system performs as it is intended to. The purpose of training is to identify defects, bugs and solve any issues to ensure the project runs smoothly. Both unit testing and system testing were performed.

5.2.1. Test Cases for Unit Testing

The test cases for unit testing of both the NSL dataset collector application and NSL recognition application are done.

5.2.1.1 Data Collector Application Unit Testing

Table 6: Dataset Collector Application's Camera and Capture Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
DC-01	Initialize camera	Start capture screen	Camera is initialized successfully	Pass
DC-02	Display camera preview	Active camera	Live camera preview is displayed	Pass
DC-03	Capture image	Capture interval reached	Camera frame is captured successfully	Pass
DC-04	Detect hand	Frame containing a hand	Hand is detected and its region is identified	Pass
DC-05	Handle no hand	Frame without a hand	Frame is rejected and no image is saved	Pass
DC-06	Handle low-confidence detection	Poorly detected hand	Frame is rejected with an appropriate reason	Pass
DC-07	Handle partial hand	Hand partially outside frame	Frame is rejected	Pass
DC-08	Crop detected hand	Valid detected hand	Hand region is cropped correctly	Pass
DC-09	Save cropped image	Valid cropped hand	Cropped image is saved under the selected label	Pass

DC-10	Save original image	Successfully captured frame	Original image is retained for future processing	Pass

Table 7: Dataset Collector Application's Label and Dataset Management Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
DC-11	Select sign label	Select a Devanagari character	Selected label is displayed for collection	Pass
DC-12	Display reference sign	Open a label	Reference image and character information are displayed	Pass
DC-13	Start timed capture	Select capture interval and target	Capture session starts with countdown	Pass
DC-14	Pause capture	Press Pause	Capture process is paused	Pass
DC-15	Resume capture	Press Resume	Capture process continues	Pass
DC-16	Stop capture	Press Stop	Capture session terminates safely	Pass
DC-17	Reach target count	Number of samples reaches target	Capture session completes automatically	Pass
DC-18	Update collection count	Successfully save image	Label count and progress are updated	Pass
DC-19	Preview collected images	Open dataset preview	Images collected for the selected label are displayed	Pass
DC-20	Delete label dataset	Select Delete	Images and associated records for the label are removed	Pass

Table 8: Dataset Collector Application's Metadata, Settings and Export Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
DC-21	Save metadata	Successfully save image	Corresponding image information is added to metadata	Pass
DC-22	Persist capture settings	Change capture settings	Updated settings are retained for later sessions	Pass
DC-23	Change capture interval	Select 2, 3, or 5 seconds	Selected interval is applied to capture	Pass
DC-24	Change camera	Select front/back camera	Selected camera is used for capture	Pass
DC-25	Change image quality	Modify image quality	Selected quality is applied when saving originals	Pass
DC-26	Export dataset	Select Export Dataset	Dataset is packaged into a ZIP file	Pass
DC-27	Share exported dataset	Complete dataset export	Android share interface is opened	Pass
DC-28	Handle insufficient storage	Start capture with insufficient storage	System warns the user and prevents unsafe capture	Pass

5.2.1.2 Recognition Application Unit Testing

Table 9: Recognition Application's Camera and Media Pipe Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
RC-01	Initialize camera	Start recognition	Camera is initialized successfully	Pass
RC-02	Display camera preview	Active camera	Live camera preview is displayed	Pass
RC-03	Capture frame	Live camera stream	Frames are continuously captured for recognition	Pass
RC-04	Detect hand	Frame containing a hand	Media Pipe detects the hand successfully	Pass
RC-05	Extract landmarks	Detected hand	Hand landmark information is extracted	Pass
RC-06	Generate landmark features	Detected hand landmarks	Required landmark coordinate features are generated	Pass
RC-07	Handle no hand	Frame without a hand	No prediction is generated	Pass
RC-08	Handle camera error	Camera unavailable/permission denied	Appropriate error message is displayed	Pass

Table 10: Recognition Application's MLP Classification Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
RC-09	Load MLP model	Start recognition	MLP model is loaded successfully	Pass
RC-10	Prepare landmark input	Extracted hand landmarks	Correct feature vector is prepared for the MLP	Pass
RC-11	Recognize valid character	Landmark features of a valid NSL sign	Corresponding character and confidence are returned	Pass
RC-12	Handle uncertain prediction	Ambiguous hand sign	Prediction is not immediately confirmed	Pass
RC-13	Handle missing landmark data	Incomplete landmark information	Invalid input is rejected safely	Pass
RC-14	Handle model error	Invalid/unavailable model	Appropriate recognition error is reported	Pass

Table 11: Recognition Application's Prediction Stabilization and Sentence Processing

ID	Description	Input	Expected Output	Pass/Fail
RC-15	Confirm stable prediction	Same character predicted across consecutive frames	Character is confirmed after required stability	Pass
RC-16	Reject unstable prediction	Different characters predicted consecutively	No unstable character is confirmed	Pass
RC-17	Reject low-confidence prediction	Prediction below confidence threshold	Prediction is ignored by the stabilizer	Pass
RC-18	Detect character change	Character changes from one sign to another	Stability tracking starts with the new character	Pass
RC-19	Preventing repeated character	Same sign held continuously	Character is not repeatedly added	Pass
RC-20	Add confirmed character	Stable valid prediction	Character is added to the text buffer	Pass
RC-21	Detect word boundary	Appropriate pause between signs	Word boundary is identified	Pass
RC-22	Detect sentence boundary	Longer signing pause	Sentence/utterance boundary is identified	Pass
RC-23	Buffer characters	Multiple confirmed characters	Characters are accumulated correctly	Pass
RC-24	Clear text buffer	Select Clear	Current recognized text is removed	Pass
RC-25	Undo last character	Select Undo	Most recent input is removed	Pass

Table 12: Recognition Application's Text and Speech Unit Testing

ID	Description	Input	Expected Output	Pass/Fail
RC-26	Display recognized character	Confirmed character	Character is displayed on screen	Pass
RC-27	Display recognized text	Sequence of confirmed characters	Corresponding Nepali text is displayed	Pass
RC-28	Generate sentence	Buffered character sequence	Text is combined into the intended sentence format	Pass
RC-29	Handle empty input	Empty text buffer	No unnecessary sentence is generated	Pass
RC-30	Speak recognized text	Select Speak	Recognized Nepali text is converted into speech	Pass
RC-31	Handle unavailable voice	Unsupported Devanagari voice	Appropriate speech availability message is shown	Pass
RC-32	Replace active speech	Start speaking and request another sentence	Previous speech stops and new speech begins	Pass

5.2.2. Test Cases for System Testing

Table 13: Dataset Collector Application's System Testing

ID	Description	Input	Expected Output	Pass/Fail
ST-DC-01	Complete data collection	Select a character and start capture	Images are captured, detected, cropped, labelled, and stored correctly	Pass
ST-DC-02	Quality-based collection	Provide valid and invalid hand frames	Valid frames are saved while unsuitable frames are rejected	Pass
ST-DC-03	Timed collection	Set target and capture interval	Images are collected automatically until target is reached	Pass
ST-DC-04	Pause and resume collection	Pause and resume during capture	Collection stops and continues without corrupting the session	Pass
ST-DC-05	Dataset preview and deletion	Collect images, preview, then delete	Images are displayed correctly and removed when requested	Pass
ST-DC-06	Dataset export	Complete collection and export	Complete dataset is packaged and shared successfully	Pass
ST-DC-07	Settings integration	Change capture settings and start collection	Selected settings are applied correctly	Pass

Table 14: Recognition Application Application's System Testing

ID	Description	Input	Expected Output	Pass/Fail
ST-RC-01	Start recognition	Open application and start recognition	Camera and recognition components initialize successfully	Pass
ST-RC-02	Complete character recognition	Present a valid NSL character	Media Pipe detects the hand, landmarks are extracted, and MLP predicts the character	Pass
ST-RC-03	Continuous recognition	Present multiple characters sequentially	Characters are recognized and accumulated correctly	Pass
ST-RC-04	Prediction stabilization	Hold a character steadily	Character is confirmed without repeated output	Pass
ST-RC-05	Invalid hand input	Remove hand or provide unclear sign	System does not add an incorrect character	Pass
ST-RC-06	Sentence formation	Sign multiple characters with pauses	Characters are combined into appropriate words and text	Pass
ST-RC-07	Text-to-speech	Generate recognized Nepali text and select Speak	Recognized text is spoken through the device	Pass
ST-RC-08	Stop recognition	Start and then stop recognition	Camera and recognition resources are released correctly	Pass
ST-RC-09	Camera permission handling	Deny camera permission	Application handles the error without crashing	Pass
ST-RC-10	End-to-end recognition	Sign a sequence of characters from start to speech output	System detects the hand, classifies signs, stabilizes predictions, forms text, and produces speech	Pass

5.2.3 Test Results

5.2.3.1 Character level NSL recognition

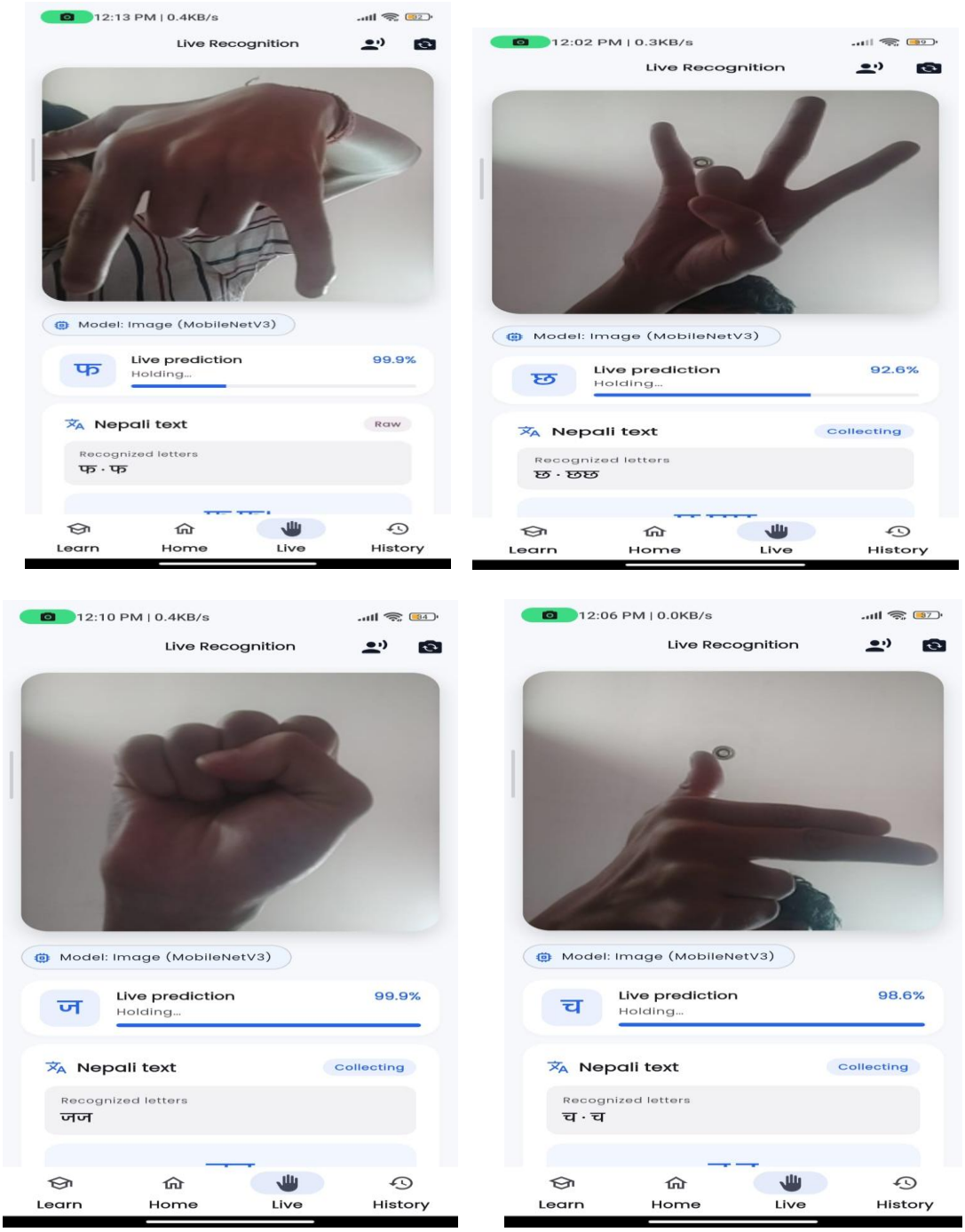


Figure 13: Character level NSL recognition

5.2.3.2 Word level NSL recognition

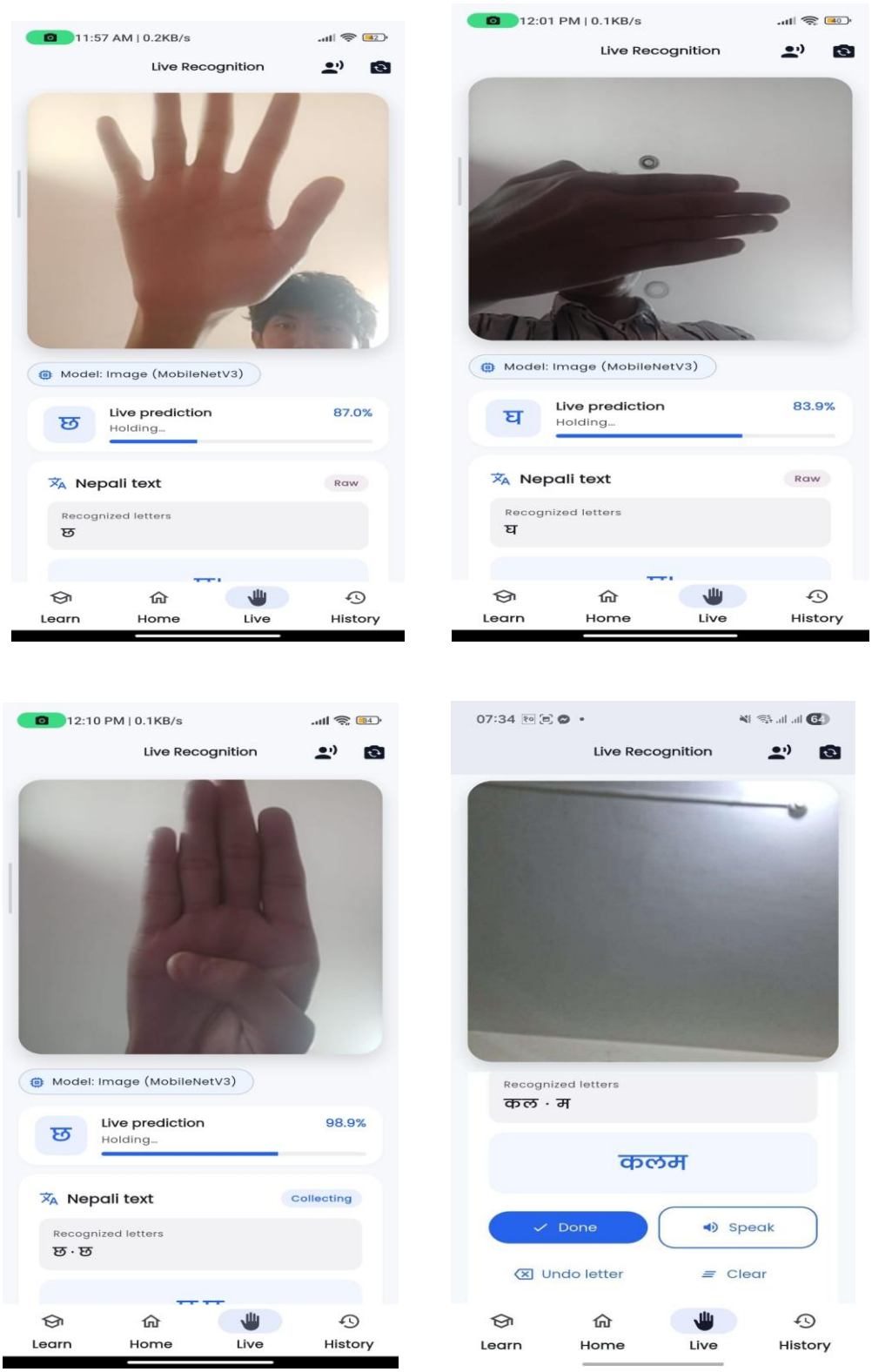


Figure 14: Word level NSL recognition

5.3. Result Analysis

The Nepali Sign Language recognition system was evaluated on a test dataset containing 36 NSL consonant classes. Media Pipe Hands was used for hand detection and landmark detection, followed by a Multilayer Perceptron for sign classification.

The model performance was evaluated using a confusion matrix and training/validation loss and accuracy.

5.3.1 Media Pipe Hand Detection

Hand detection and localization, the first stage of the recognition pipeline, is based on Media Pipe Hands. It detects the hand within the image frame and returns the corresponding hand landmarks that need to be processed into the input feature vector.

The detected hand landmarks are preprocessed and normalized before being passed to the classification model. This separation of hand detection from classification allows the MLP classifier to focus purely on the relevant landmark-based hand information, rather than raw image data.

5.3.2 Multilayer Perceptron

The Multilayer Perceptron is used as the classification model for recognizing the Nepali Sign Language consonants. Media Pipe's hand landmarks are fed to the MLP which is trained to recognize the pattern of hand landmarks and make a prediction of the NSL consonant class.

To measure the classification ability of the model, a confusion matrix, training accuracy and validation accuracy and loss were used on 36 NSL consonant classes.

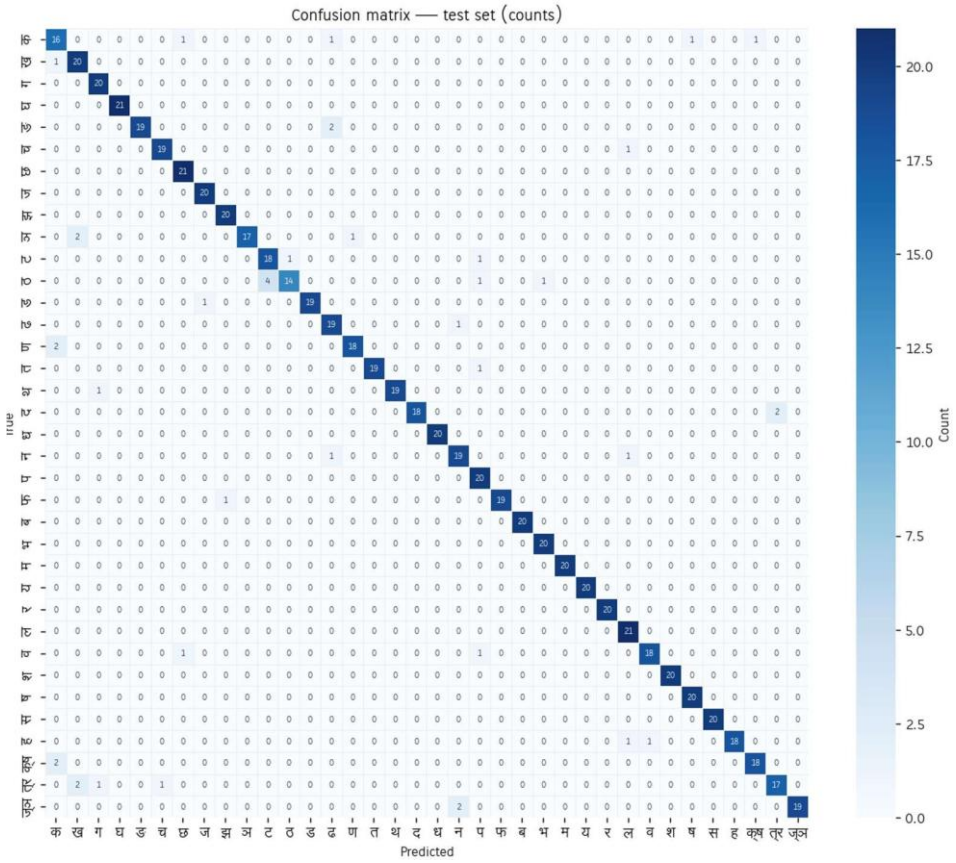


Figure 15: Confusion Matrix

The confusion matrix shows a strong concentration of values along the main diagonal. This indicates that the MLP model correctly recognizes a large proportion of the test samples for the corresponding NSL classes. Most classes have high diagonal counts, while only a small number of samples appear outside the diagonal.

The off-diagonal values indicate occasional confusion between visually similar hand signs. Such misclassifications are expected in sign-language recognition because some consonants may have similar hand configurations or differ only in subtle hand/finger positions.

Overall, the confusion matrix demonstrates that the trained MLP model is capable of distinguishing the majority of the 36 NSL consonant classes in the test dataset. The class-wise confusion matrix is particularly useful for identifying which signs require further improvement through additional training samples, improved preprocessing, or better feature representation.

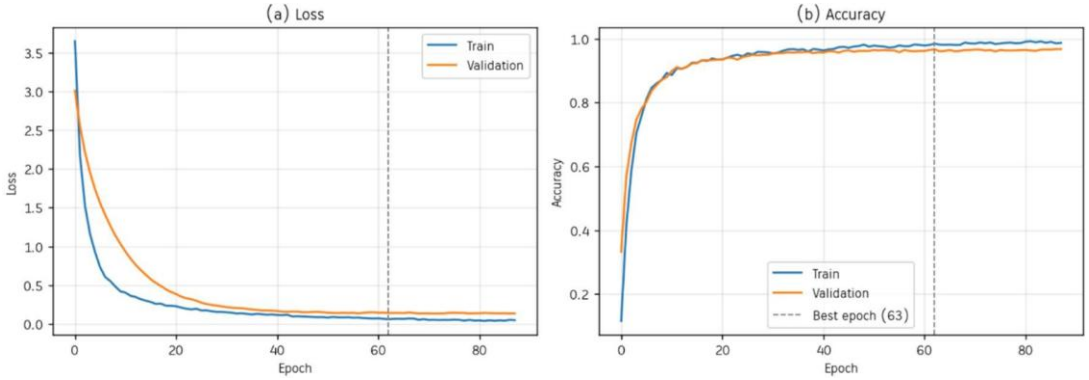


Figure 16: Model Performance

The loss graph shows a significant drop in training loss and validation loss in the first few epochs. epochs. The training loss reduces from ~3.6 to 0.05 and the validation loss is reduced from approximately 0.7 to 0.05.The loss of approximately 3.0 is lowered to 0.15. Both curves become after the initial drop. A relatively stable reading which means that the model has converged gradually towards a stable state.

The model performance is calculated as follows:

Table 15: Overall Model Performance

Metric	Value
Accuracy	94.23%
Precision	94.61%
Recall	94.24%
F1 score	94.22%

The training and validation curves are not too far apart, suggesting that the model is well generalizable. performs in a stable manner on both datasets. Overall, the results are indicative that the NSL consonant classes were successfully learned by the MLP model, and the stable and high accuracy was obtained classification performance.

CHAPTER 6 : CONCLUSION AND FUTURE RECOMMENDATION

6.1. Conclusion

The project “SignSathi: Real-Time Nepali Sign Language Recognition” was designed to enable a simple and accessible tool to recognize NSL fingerspelling and get the translated text and speech in Nepali. This system is able to recognize 36 Devanagari consonant characters from the live camera feed, and it displays the recognized characters.

The first step in the recognition process is hand detection by Media Pipe, which extracts 21 landmarks from the hand. The landmarks are specified by their X, Y, and Z coordinates, resulting in 63 landmark coordinates that specify hand position and shape. The extracted features are fed into an MLP model, which is used to predict the corresponding NSL character. To obtain a stable output during real-time recognition, prediction stabilization is employed, in which the characters are confirmed only when the prediction is consistent in consecutive frames. The characters identified are then assembled to make words and sentences that can be presented as Nepali script and played back as speech.

Overall, the main aim of the project of real-time Nepali Sign Language recognition system is achieved and it has been able to recognize the supported fingerspelling of NSL and give the text and speech output without relying on any server or continuous internet connection. The project also provides an example of the application of computer vision and machine learning to solve a real communication problem.

However, the system still has some limitations. Its recognition performance can be affected by factors such as poor lighting, unsuitable hand positioning, and complex backgrounds. The current implementation is also limited to single-hand static fingerspelling and the supported set of 36 Devanagari consonant characters. Therefore, future improvements could focus on expanding the range of recognized sig ns and making the system more robust under different real-world conditions.

6.2 Future Recommendation

The system works well within its current scope, but there are several directions for future improvement:

i. Expanded Character Set: Extending recognition to include Devanagari vowels, numerals, and common word-level signs, beyond the current 36 consonant characters.
ii. Dynamic Sign Support: Incorporating temporal/movement-based modeling (e.g., sequence models) to recognize dynamic signs that involve hand motion.
iii. Two-Handed Sign Recognition: Extending the hand detection and classification pipeline to support signs that require both hands.
iv. Improved Dataset Diversity: Collecting a more diverse dataset with varied lighting conditions, backgrounds, hand sizes, and skin tones to make the model more robust in real-world environments.
v. Robustness to Camera Conditions: Improving hand detection and preprocessing to maintain accuracy under poor lighting or difficult backgrounds.

REFERENCES

[1] R. Rastgoo, K. Kiani, and S. Escalera, "Sign language recognition: A deep survey," *Expert Systems with Applications*, vol. 164, Art. no. 113794, 2021.

[2] I. Alam, A. Hameed, and R. A. Ziar, "Exploring sign language detection on smartphones: A systematic review of machine and deep learning approaches," *Advances in Human-Computer Interaction*, vol. 2024, Art. no. 1487500, 2024.

[3] H. R. V. Joze and O. Koller, "MS-ASL: A large-scale data set and benchmark for understanding American Sign Language," in *Proc. British Machine Vision Conference (BMVC)*, 2019.

[4] D. Li, C. Rodriguez, X. Yu, and H. Li, "Word-level deep sign language recognition from video: A new large-scale dataset and methods comparison," in *Proc. IEEE Winter Conf. on Applications of Computer Vision (WACV)*, 2020.

[5] A. Desai *et al.*, "ASL Citizen: A community-sourced dataset for advancing isolated sign language recognition," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2023.

[6] F. Zhang *et al.*, "MediaPipe Hands: On-device real-time hand tracking," arXiv:2006.10214, 2020.

[7] M. De Coster, E. Rushe, R. Holmes, A. Ventresque, and J. Dambre, "Towards the extraction of robust sign embeddings for low resource sign language recognition," arXiv:2306.17558, 2023.

[8] Y. Zhang and G. Notni, "3D geometric features based real-time American sign language recognition using PointNet and MLP with MediaPipe hand skeleton detection," *Measurement: Sensors*, vol. 38, Art. no. 101697, 2025.

[9] M. Z. Uddin, C. Boletsis, and P. Rudshavn, "Real-time Norwegian sign language recognition using MediaPipe and LSTM," *Multimodal Technologies and Interaction*, vol. 9, no. 3, Art. no. 23, 2025.

[10] D. Rathi, "Optimization of transfer learning for sign language recognition targeting mobile platform," arXiv:1805.06618, 2018.

[11] Y. Li, B. Ren, K. Hu, C. Liu, Z. Jiang, K. Dang, and J. Su, "KD-MSLRT: Lightweight sign language recognition model based on MediaPipe and 3D to 1D knowledge distillation," arXiv:2501.02321, 2025.

[12] A. B. Varanasi, M. Sinha, and T. Dasgupta, "Isolated sign language recognition via MediaPipe landmarks: A case study on Indian Sign Language," in *Proc. IEEE/CVF Conf. on Computer Vision and Pattern Recognition (CVPR) Workshops*, pp. 10595–10604, 2026.

[13] M. Acharya and D. R. Pant, "Computer vision based hand gesture recognition for speech disabled persons," *Journal of the Institute of Engineering*, vol. 11, no. 1, pp. 30–35, 2015.

[14] D. Mali, R. Mali, S. Sipai, and S. P. Panday, "Two dimensional (2D) convolutional neural network for Nepali sign language recognition," in *Proc. 12th Int. Conf. on Software, Knowledge, Information Management and Applications (SKIMA)*, IEEE, pp. 1–5, 2018.

[15] V. Thapa, J. Sunuwar, and R. Pradhan, "Finger spelling recognition for Nepali sign language," in *Recent Developments in Machine Learning and Data Analytics*, Advances in Intelligent Systems and Computing, vol. 740, pp. 219–227, Springer, 2019.

[16] S. Ligal and D. S. Baral, "Nepali sign language gesture recognition using deep learning," in *Proc. 12th IOE Graduate Conference*, vol. 12, pp. 1788–1795, 2022.

[17] S. Heuju and B. Rijal, "Word level Nepali sign language recognition using Transformer," in *Proc. 12th IOE Graduate Conference*, vol. 12, 2022.

[18] G. Shrestha, N. Thing, P. Dhakal, P. Dahal, P. Karki, and M. K. Guragain, "Nepali sign language letter detection and finger spelling using MediaPipe and CNN," in *Proc. 15th IOE Graduate Conference*, Institute of Engineering, Tribhuvan University, pp. 166–170, 2024.

[19] J. Sunuwar, S. Borah, and A. Kharga, "NSL23 dataset for alphabets of Nepali sign language," *Data in Brief*, 2024.

[20] D. Bhusal and D. S. Baral, "Sentence-level Nepali sign language recognition," in *Proc. 2024 IEEE Int. Conf. on Inventive Computation Technologies (ICICT)*, pp. 972–979, 2024.

[21] B. Poudel, S. Ghimire, S. Bhattarai, S. Bhandari, and S. Sharma Dahal, “Nepali sign language characters recognition: Dataset development and deep learning approaches,” *KEC Journal of Science and Engineering*, vol. 10, no. 1, pp. 131–136, 2026.